



Estoy desarrollando un chatbot de gestión de vacaciones en Python.
¿Cómo debería dividir el programa en funciones para que el código
quede ordenado y fácil de mantener?

Organización del Código para Chatbot de Gestión de Vacaciones

Basado en los requisitos de tu Trabajo Práctico Integrador, te propongo una estructura modular que refleje tanto el modelo BPMN como los principios de código limpio y mantenible.

Estructura de Directorios Recomendada

text

```
chatbot-vacaciones/  
├── src/  
│   ├── __init__.py  
│   ├── main.py                # Punto de entrada  
│   └── bot/  
│       ├── __init__.py  
│       ├── handlers.py        # Manejadores de mensajes  
│       └── state_machine.py    # Gestión de estados  
│   ├── core/  
│       ├── __init__.py  
│       ├── bpmn_engine.py      # Lógica basada en BPMN  
│       ├── gateways.py         # Decisiones y compuertas  
│       └── validators.py        # Validaciones de entrada  
│   ├── data/  
│       ├── __init__.py  
│       ├── database.py         # Interacción con datos  
│       └── models.py           # Definición de entidades  
│   ├── config/  
│       ├── __init__.py  
│       └── settings.py         # Configuraciones  
│   └── utils/  
│       ├── __init__.py  
│       └── logger.py           # Logging y debugging  
├── data/  
│   ├── empleados.xlsx          # Datos simulados  
│   └── vacaciones.xlsx  
├── tests/  
│   ├── test_bpmn.py  
│   └── test_state_machine.py  
├── docs/  
│   ├── BPMN_diagram.png  
│   ├── diccionario_datos.md  
│   └── manual_usuario.md  
├── .env  
├── requirements.txt  
└── README.md
```

Desglose de Funciones por Módulo

1. Módulo de Gestión de Estados (bot/state_machine.py)

python

```

class VacationStateMachine:
    """Máquina de estados del proceso de vacaciones (BPMN)"""

    STATES = {
        'INICIO': 'inicio',
        'INGRESO_LEGajo': 'ingreso_legajo',
        'VALIDAR_LEGajo': 'validar_legajo',
        'INGRESO_FECHAS': 'ingreso_fechas',
        'VALIDAR_FECHAS': 'validar_fechas',
        'VERIFICAR_SALDO': 'verificar_saldo',
        'DECISION_SALDO': 'decision_saldo', # Gateway
        'APROBACION_JEFE': 'aprobacion_jefe',
        'DECISION_JEFE': 'decision_jefe', # Gateway
        'CONFIRMACION': 'confirmacion',
        'FIN_APROBADO': 'fin_aprobado',
        'FIN_RECHAZADO': 'fin_rechazado'
    }

    def __init__(self):
        self.user_states = {} # {user_id: estado_actual}
        self.user_data = {} # {user_id: {datos_acumulados}}

    def get_state(self, user_id: str) -> str:
        return self.user_states.get(user_id, self.STATES['INICIO'])

    def transition_to(self, user_id: str, new_state: str) -> bool:
        """Valida y realiza la transición de estado"""
        if new_state not in self.STATES.values():
            raise ValueError(f"Estado inválido: {new_state}")
        self.user_states[user_id] = new_state
        return True

    def save_data(self, user_id: str, key: str, value: any) -> None:
        """Persiste datos del usuario en el estado actual"""
        if user_id not in self.user_data:
            self.user_data[user_id] = {}
        self.user_data[user_id][key] = value

    def get_data(self, user_id: str, key: str) -> any:
        return self.user_data.get(user_id, {}).get(key)

```

2. Módulo de Handlers (bot/handlers.py)

```

python

from .state_machine import VacationStateMachine
from core.validators import InputValidator
from data.database import DataManager

class MessageHandler:
    """Maneja los mensajes entrantes y coordina el flujo BPMN"""

    def __init__(self, state_machine: VacationStateMachine):
        self.state_machine = state_machine
        self.validator = InputValidator()
        self.db = DataManager()
        self.bpmn_flow = BPMNFlowHandler(state_machine, self.db)

    def handle_message(self, user_id: str, message: str) -> str:
        """Punto de entrada principal - Patrón Chain of Responsibility"""
        current_state = self.state_machine.get_state(user_id)

        # Mapeo estado -> manejador específico
        handlers = {
            'inicio': self._handle_start,
            'ingreso_legajo': self._handle_legajo,

```

```

        'validar_legajo': self._handle_validate_legajo,
        'ingreso_fechas': self._handle_fechas,
        'validar_fechas': self._handle_validate_fechas,
        'verificar_saldo': self._handle_verify_saldo,
        'decision_saldo': self._handle_saldo_decision,
        'aprobacion_jefe': self._handle_jefe_approval,
        'decision_jefe': self._handle_jefe_decision,
        'confirmacion': self._handle_confirm
    }

    handler = handlers.get(current_state, self._handle_unknown)
    return handler(user_id, message)

def _handle_start(self, user_id: str, message: str) -> str:
    """Inicio del flujo BPMN - Evento de inicio"""
    self.state_machine.transition_to(user_id, 'ingreso_legajo')
    return "📄 Por favor, ingresa tu número de legajo:"

def _handle_legajo(self, user_id: str, message: str) -> str:
    """Tarea de usuario: Ingreso de legajo"""
    if self.validator.validate_legajo(message):
        self.state_machine.save_data(user_id, 'legajo', message)
        self.state_machine.transition_to(user_id, 'validar_legajo')
        return self._handle_validate_legajo(user_id, message)
    else:
        return "❌ Legajo inválido. Debe ser numérico de 6 dígitos. Reintenta:"

def _handle_validate_legajo(self, user_id: str, legajo: str) -> str:
    """Tarea de sistema: Validar legajo en base de datos"""
    empleado = self.db.get_empleado_by_legajo(legajo)
    if empleado:
        self.state_machine.save_data(user_id, 'empleado', empleado)
        self.state_machine.transition_to(user_id, 'ingreso_fechas')
        return f"✅ Empleado encontrado: {empleado['nombre']}\n📅 Ingresa fecha inicio (DD/MM/AAAA):"
    else:
        self.state_machine.transition_to(user_id, 'fin_rechazado')
        return "❌ Legajo no encontrado. Fin del proceso."

```

3. Módulo de Gateway (core/gateways.py)

python

```

class BPMNGateways:
    """Implementación de las compuertas (gateways) del diagrama BPMN"""

    @staticmethod
    def decision_saldo(user_data: dict) -> dict:
        """
        Gateway XOR: ¿Tiene saldo suficiente?
        Implementa el punto de decisión del diagrama BPMN
        """
        saldo_disponible = user_data.get('saldo_disponible', 0)
        dias_solicitados = user_data.get('dias_solicitados', 0)

        return {
            'aprobado': dias_solicitados <= saldo_disponible,
            'saldo_restante': saldo_disponible - dias_solicitados,
            'mensaje': f"Saldo disponible: {saldo_disponible} días\n"
                       f"Días solicitados: {dias_solicitados} días"
        }

    @staticmethod
    def decision_jefe(user_data: dict) -> dict:
        """
        Gateway XOR: ¿Aprobación del jefe?
        """

```

```

return {
    'aprobado': user_data.get('aprobacion_jefe', False),
    'motivo': user_data.get('motivo_rechazo', 'No especificado')
}

```

4. Módulo de Validadores (core/validators.py)

python

```

import re
from datetime import datetime

class InputValidator:
    """Valida las entradas del usuario (Camino Feliz e Infeliz)"""

    @staticmethod
    def validate_legajo(legajo: str) -> bool:
        """Validación del número de legajo"""
        pattern = r'^[0-9]{6}$'
        return bool(re.match(pattern, legajo.strip()))

    @staticmethod
    def validate_fecha(fecha: str) -> dict:
        """
        Valida fecha en formato DD/MM/AAAA
        Retorna dict con estado y mensaje
        """
        try:
            fecha_dt = datetime.strptime(fecha, '%d/%m/%Y')
            fecha_actual = datetime.now()

            # Validación negocio: no puede ser en pasado
            if fecha_dt < fecha_actual:
                return {
                    'valid': False,
                    'message': '❌ La fecha no puede ser anterior a hoy'
                }

            return {
                'valid': True,
                'data': fecha_dt,
                'message': '✅ Fecha válida'
            }
        except ValueError:
            return {
                'valid': False,
                'message': '❌ Formato inválido. Usa DD/MM/AAAA'
            }

    @staticmethod
    def validate_cantidad_dias(dias: str) -> dict:
        """Valida que sea un número entero positivo"""
        try:
            num_dias = int(dias)
            if num_dias <= 0:
                return {
                    'valid': False,
                    'message': '❌ Los días deben ser mayores a 0'
                }
            if num_dias > 30:
                return {
                    'valid': False,
                    'message': '❌ Máximo 30 días consecutivos'
                }
            return {'valid': True, 'data': num_dias}
        except ValueError:
            return {

```

```

        'valid': False,
        'message': '❌ Debes ingresar un número válido'
    }

```

5. Módulo de Datos (data/database.py)

python

```

import pandas as pd
from typing import Optional, Dict

class DataManager:
    """Capa de persistencia - Simula la base de datos"""

    def __init__(self, empleados_path: str = 'data/empleados.xlsx'):
        self.empleados = self._load_empleados(empleados_path)
        self.vacaciones = {} # Simula tabla de vacaciones

    def _load_empleados(self, path: str) -> Dict:
        """Carga datos desde Excel"""
        try:
            df = pd.read_excel(path)
            return df.to_dict('records')
        except FileNotFoundError:
            # Datos de ejemplo para simulación
            return [
                {'legajo': '123456', 'nombre': 'Juan Pérez', 'saldo_vacaciones': 15, 'jefe': 'jefe@empresa.com'},
                {'legajo': '789012', 'nombre': 'María Gómez', 'saldo_vacaciones': 10, 'jefe': 'jefe@empresa.com'}
            ]

    def get_empleado_by_legajo(self, legajo: str) -> Optional[Dict]:
        """Tarea de sistema: Consulta de datos"""
        for emp in self.empleados:
            if emp['legajo'] == legajo:
                return emp
        return None

    def registrar_solicitud(self, user_id: str, data: Dict) -> bool:
        """Persistencia de la solicitud"""
        self.vacaciones[user_id] = {
            'fecha_solicitud': pd.Timestamp.now(),
            'datos': data,
            'estado': 'pendiente'
        }
        return True

```

Punto de Entrada Principal (main.py)

python

```

from src.bot.state_machine import VacationStateMachine
from src.bot.handlers import MessageHandler
from src.data.database import DataManager

def main():
    """Configuración e inicio del bot"""

    # Inicialización
    state_machine = VacationStateMachine()
    db = DataManager()
    handler = MessageHandler(state_machine, db)

    # Simulación de chat

```

```
def simular_conversacion():
    user_id = "test_user_001"
    print("🤖 Bot de Gestión de Vacaciones iniciado")

    while True:
        user_input = input("\n👤 Tú: ")
        if user_input.lower() in ['salir', 'exit']:
            break

        response = handler.handle_message(user_id, user_input)
        print(f"\n🤖 Bot: {response}")

    simular_conversacion()

if __name__ == "__main__":
    main()
```

 Relación con el Diagrama BPMN

Componente Código	Elemento BPMN
VacationStateMachine	Estados del proceso
_handle_legajo	Tarea de Usuario
_handle_validate_legajo	Tarea de Servicio
BPMNGateways.decision_saldo	Compuerta XOR
DataManager.get_empleado	Consulta a Base de Datos
Validadores	Manejo de Excepciones

 Consejos para tu Entrega

- 1. **Diccionario de Datos:** Documenta todas las variables en docs/diccionario_datos.md
- 2. **Manejo del Camino Infeliz:** Cada validator retorna mensajes de error claros
- 3. **Pruebas:** Incluye casos de prueba para cada gateway
- 4. **Git:** Commits por cada módulo implementado

Esta estructura te permite:

-  Mantener separación de responsabilidades
-  Facilitar pruebas unitarias
-  Escalar a más funcionalidades
-  Cumplir con los requisitos BPMN del TFI

¿Necesitas que profundice en algún módulo específico o que te ayude con la documentación requerida?

Necesito validar el legajo, la fecha de inicio y la cantidad de días solicitados. ¿Qué validaciones me recomendás implementar y cómo debería manejar los errores de ingreso del usuario?

Validaciones y Manejo de Errores para Gestión de Vacaciones

Te propongo un sistema completo de validaciones que cubre todos los casos posibles (Camino Feliz e Infeliz) y cumple con los requisitos de robustez del TFI.

1. Módulo de Validaciones Mejorado (core/validators.py)

```
python

import re
from datetime import datetime, date
from typing import Tuple, Dict, Optional, Union
from enum import Enum

class ValidationResult:
    """Estandariza los resultados de validación"""

    def __init__(self, valid: bool, data: any = None,
                  error_code: str = None, message: str = None):
        self.valid = valid
        self.data = data
        self.error_code = error_code
        self.message = message or self._get_default_message(error_code)

    def _get_default_message(self, code: str) -> str:
        """Mensajes predefinidos por tipo de error"""
        messages = {
            'EMPTY': '⚠ El campo no puede estar vacío',
            'INVALID_FORMAT': '❌ Formato incorrecto',
            'NOT_FOUND': '❌ No encontrado',
            'INVALID_LENGTH': '❌ Longitud incorrecta',
            'INVALID_RANGE': '❌ Valor fuera de rango',
            'PAST_DATE': '❌ La fecha no puede ser anterior a hoy',
            'INSUFFICIENT_DAYS': '❌ Saldo insuficiente',
            'MAX_EXCEEDED': '❌ Supera el máximo permitido',
            'HOLIDAY': '⚠ Fecha en día no laborable'
        }
        return messages.get(code, '❌ Dato inválido')

class LegajoValidator:
    """Validaciones específicas para número de legajo"""

    @staticmethod
    def validate(legajo: str) -> ValidationResult:
        """
        Validación completa del legajo
        Retorna: ValidationResult con estado y datos
        """
        # 1. Validación de entrada vacía
        if not legajo or not legajo.strip():
            return ValidationResult(False, error_code='EMPTY')

        legajo_clean = legajo.strip()

        # 2. Validación de formato (solo números)
        if not legajo_clean.isdigit():
            return ValidationResult(
                False,
                error_code='INVALID_FORMAT',
                message='❌ El legajo debe contener solo números'
            )

        # 3. Validación de longitud (6 dígitos)
        if len(legajo_clean) != 6:
            return ValidationResult(
                False,
                error_code='INVALID_LENGTH',
                message=f'❌ El legajo debe tener 6 dígitos (ingresaste {len(legajo_clean)})'
            )

        # 4. Validación de negocio: Legajo no puede ser todo ceros
        if legajo_clean == '000000':

```

```

        return ValidationResult(
            False,
            error_code='INVALID_FORMAT',
            message='✖ Legajo inválido'
        )

# 5. Validación con base de datos (se hace en handler)
# Aquí solo validamos sintaxis, la existencia se valida en BD

return ValidationResult(True, data={'legajo': legajo_clean})

class FechaValidator:
    """Validaciones para fechas del proceso de vacaciones"""

    # Configuración
    DIAS_LABORABLES = [0, 1, 2, 3, 4] # Lunes a Viernes
    DIAS_FERIADOS = [] # Se cargaría desde BD

    @classmethod
    def validate_fecha_inicio(cls, fecha_str: str) -> ValidationResult:
        """Valida fecha de inicio de vacaciones"""

        # 1. Validación de entrada vacía
        if not fecha_str or not fecha_str.strip():
            return ValidationResult(False, error_code='EMPTY')

        # 2. Validación de formato DD/MM/AAAA
        try:
            fecha = datetime.strptime(fecha_str.strip(), '%d/%m/%Y').date()
        except ValueError:
            # Intentar con otros formatos comunes
            try:
                fecha = datetime.strptime(fecha_str.strip(), '%d-%m-%Y').date()
            except ValueError:
                return ValidationResult(
                    False,
                    error_code='INVALID_FORMAT',
                    message='✖ Formato inválido. Usa DD/MM/AAAA o DD-MM-AAAA'
                )

        # 3. Validación de fecha en pasado
        hoy = date.today()
        if fecha < hoy:
            return ValidationResult(
                False,
                error_code='PAST_DATE',
                message=f'✖ La fecha {fecha.strftime("%d/%m/%Y")} ya pasó'
            )

        # 4. Validación de antigüedad (no más de 6 meses adelante)
        max_fecha = hoy.replace(year=hoy.year + 1)
        if fecha > max_fecha:
            return ValidationResult(
                False,
                error_code='INVALID_RANGE',
                message='✖ Solo se pueden solicitar vacaciones con hasta 1 año de anticipación'
            )

        # 5. Validación de día laborable (opcional)
        if fecha.weekday() in [5, 6]: # Sábado o Domingo
            return ValidationResult(
                False,
                error_code='HOLIDAY',
                message=f'⚠ {fecha.strftime("%A")} no es día laborable. ¿Deseas continuar?'
            )

```

```

        return ValidationResult(True, data={'fecha': fecha})

    @classmethod
    def validate_periodo(cls, fecha_inicio: date, fecha_fin: date) -> ValidationResult:
        """Valida que el período sea válido"""

        # 1. La fecha fin no puede ser anterior a inicio
        if fecha_fin < fecha_inicio:
            return ValidationResult(
                False,
                error_code='INVALID_RANGE',
                message='❌ La fecha de fin no puede ser anterior a la de inicio'
            )

        # 2. Calcular días hábiles
        dias_totales = cls._calcular_dias_habiles(fecha_inicio, fecha_fin)

        # 3. Máximo 30 días hábiles
        if dias_totales > 30:
            return ValidationResult(
                False,
                error_code='MAX_EXCEEDED',
                message=f'❌ Máximo 30 días hábiles (solicitaste {dias_totales})'
            )

        return ValidationResult(
            True,
            data={'dias_habiles': dias_totales}
        )

    @staticmethod
    def _calcular_dias_habiles(inicio: date, fin: date) -> int:
        """Calcula días hábiles entre dos fechas"""
        dias = 0
        current = inicio
        while current <= fin:
            if current.weekday() < 5: # Lunes a Viernes
                dias += 1
                current += timedelta(days=1)
        return dias

class DiasValidator:
    """Validaciones para cantidad de días solicitados"""

    @classmethod
    def validate(cls, dias_str: str, saldo_disponible: int = None) -> ValidationResult:
        """
        Valida cantidad de días solicitados
        saldo_disponible: opcional para validar contra saldo
        """

        # 1. Validación de entrada vacía
        if not dias_str or not dias_str.strip():
            return ValidationResult(False, error_code='EMPTY')

        # 2. Validación de que sea número
        try:
            dias = int(dias_str.strip())
        except ValueError:
            return ValidationResult(
                False,
                error_code='INVALID_FORMAT',
                message='❌ Debes ingresar un número entero'
            )

        # 3. Validación de rango: mínimo 1 día
        if dias < 1:
            return ValidationResult(

```

```

        False,
        error_code='INVALID_RANGE',
        message='✖ Mínimo 1 día de vacaciones'
    )

# 4. Validación de rango: máximo 30 días
if dias > 30:
    return ValidationResult(
        False,
        error_code='MAX_EXCEEDED',
        message=f'✖ Máximo 30 días consecutivos (solicitaste {dias})'
    )

# 5. Validación contra saldo disponible
if saldo_disponible is not None:
    if dias > saldo_disponible:
        return ValidationResult(
            False,
            error_code='INSUFFICIENT_DAYS',
            message=f'✖ Saldo insuficiente. Disponible: {saldo_disponible} días'
        )

return ValidationResult(True, data={'dias': dias})

```

2. Módulo de Validación Contextual (core/context_validators.py)

python

```

from typing import Dict, List, Tuple
from .validators import ValidationResult, LegajoValidator, FechaValidator, DiasValidator

class ContextValidator:
    """
    Valida el contexto completo de la solicitud
    Integra todas las validaciones de negocio
    """

    def __init__(self, db_manager):
        self.db = db_manager

    def validate_solicitud_completa(self, user_data: Dict) -> Dict:
        """
        Valida todos los datos acumulados antes de enviar a aprobación
        Retorna: {'valid': bool, 'errors': List, 'warnings': List}
        """
        result = {
            'valid': True,
            'errors': [],
            'warnings': [],
            'data': {}
        }

# 1. Validar Legajo
legajo = user_data.get('legajo')
if legajo:
    val_legajo = LegajoValidator.validate(legajo)
    if not val_legajo.valid:
        result['valid'] = False
        result['errors'].append(val_legajo.message)

# 2. Validar fecha de inicio
fecha_inicio = user_data.get('fecha_inicio')
if fecha_inicio:
    val_fecha = FechaValidator.validate_fecha_inicio(fecha_inicio)
    if not val_fecha.valid:

```

```

        result['valid'] = False
        result['errors'].append(val_fecha.message)
    elif 'HOLIDAY' in val_fecha.error_code:
        result['warnings'].append(val_fecha.message)

# 3. Validar días
dias = user_data.get('dias_solicitados')
saldo = user_data.get('saldo_disponible')
if dias:
    val_dias = DiasValidator.validate(str(dias), saldo)
    if not val_dias.valid:
        result['valid'] = False
        result['errors'].append(val_dias.message)

return result

def validate_superposicion(self, legajo: str, fecha_inicio: date, dias: int) -> Dict:
    """
    Valida que no haya superposición con otras vacaciones
    """
    # Consultar vacaciones previas del empleado
    vacaciones_existentes = self.db.get_vacaciones_empleado(legajo)

    # Calcular rango solicitado
    fecha_fin = fecha_inicio + timedelta(days=dias-1)

    for vac in vacaciones_existentes:
        if vac['estado'] == 'aprobado':
            if not (vac['fecha_fin'] < fecha_inicio or
                    vac['fecha_inicio'] > fecha_fin):
                return {
                    'valid': False,
                    'message': f'⚠ Superposición con vacaciones del {vac["fecha_inicio"]} al {vac["fecha_fin"]}'
                }

    return {'valid': True, 'message': '✅ Sin superposiciones'}

```

3. Handler con Manejo de Errores Mejorado (bot/handlers.py)

python

```

from core.validators import LegajoValidator, FechaValidator, DiasValidator
from core.context_validators import ContextValidator
from typing import Dict

class MessageHandler:
    """Manejador con validaciones y manejo de errores robusto"""

    def __init__(self, state_machine, db_manager):
        self.state_machine = state_machine
        self.db = db_manager
        self.context_validator = ContextValidator(db_manager)
        self.max_retries = 3 # Intentos permitidos por campo
        self._initialize_retry_counters()

    def _initialize_retry_counters(self):
        """Inicializa contadores de reintentos por usuario"""
        self.retry_count = {}

    def handle_message(self, user_id: str, message: str) -> str:
        """Punto de entrada con manejo de errores global"""
        try:
            current_state = self.state_machine.get_state(user_id)

            # Mapeo de estados con sus handlers

```

```

handlers = {
    'ingreso_legajo': self._handle_legajo_with_retry,
    'ingreso_fechas': self._handle_fechas_with_retry,
    'ingreso_dias': self._handle_dias_with_retry,
    'validar_periodo': self._handle_periodo_validation,
    'verificar_saldo': self._handle_saldo_validation
}

handler = handlers.get(current_state, self._handle_unknown)
return handler(user_id, message)

except Exception as e:
    # Captura de errores inesperados
    return f"❌ Error inesperado: {str(e)}\nPor favor, intenta de nuevo"

def _handle_legajo_with_retry(self, user_id: str, legajo: str) -> str:
    """
    Maneja la validación del legajo con reintentos
    """
    # Verificar límite de reintentos
    if self.retry_count.get(user_id, 0) >= self.max_retries:
        self._reset_user_session(user_id)
        return f"❌ Demasiados intentos fallidos. Inicia nuevamente el proceso."

    # Validar formato
    result = LegajoValidator.validate(legajo)

    if not result.valid:
        # Incrementar contador de errores
        self.retry_count[user_id] = self.retry_count.get(user_id, 0) + 1
        intentos_restantes = self.max_retries - self.retry_count[user_id]

        return f"{result.message}\n🔄 Intentos restantes: {intentos_restantes}"

    # Validar existencia en BD
    empleado = self.db.get_empleado_by_legajo(result.data['legajo'])
    if not empleado:
        self.retry_count[user_id] = self.retry_count.get(user_id, 0) + 1
        return f"❌ Legajo {result.data['legajo']} no encontrado en el sistema\n"

    # Éxito: guardar datos y avanzar
    self.state_machine.save_data(user_id, 'legajo', result.data['legajo'])
    self.state_machine.save_data(user_id, 'empleado', empleado)
    self.state_machine.transition_to(user_id, 'ingreso_fechas')

    # Resetear contador de errores
    self.retry_count[user_id] = 0

    return f"✅ Empleado: {empleado['nombre']}\n📅 Ingresa la fecha de inicio (DD/MM/AAAA):"

def _handle_fechas_with_retry(self, user_id: str, fecha_str: str) -> str:
    """Maneja validación de fecha con reintentos"""

    # Validar formato y rango
    result = FechaValidator.validate_fecha_inicio(fecha_str)

    if not result.valid:
        # Caso especial: advertencia de día no laborable
        if result.error_code == 'HOLIDAY':
            # Ofrecer opción de continuar
            return f"{result.message}\n👉 Responde 'si' para continuar o 'no' para cambiar:"

        # Error normal
        self.retry_count[user_id] = self.retry_count.get(user_id, 0) + 1
        return f"{result.message}\n🔄 Intenta nuevamente (DD/MM/AAAA):"

```

```

# Guardar fecha
self.state_machine.save_data(user_id, 'fecha_inicio', result.data['fecha'])
self.state_machine.transition_to(user_id, 'ingreso_dias')

return f"✅ Fecha: {result.data['fecha'].strftime('%d/%m/%Y')}\n 📅 ¿Cuántos días solicitás?"

def _handle_dias_with_retry(self, user_id: str, dias_str: str) -> str:
    """Maneja validación de días con reintentos"""

    # Obtener datos del usuario
    user_data = self.state_machine.get_all_data(user_id)
    saldo = user_data.get('empleado', {}).get('saldo_vacaciones', 0)

    # Validar días
    result = DiasValidator.validate(dias_str, saldo)

    if not result.valid:
        self.retry_count[user_id] = self.retry_count.get(user_id, 0) + 1
        return f"{result.message}\n 📅 Reingresa la cantidad de días:"

    # Guardar días
    dias = result.data['dias']
    self.state_machine.save_data(user_id, 'dias_solicitados', dias)
    self.state_machine.transition_to(user_id, 'validar_periodo')

    # Calcular fecha fin para validar período
    fecha_inicio = user_data.get('fecha_inicio')
    fecha_fin = fecha_inicio + timedelta(days=dias-1)

    return self._handle_periodo_validation(user_id, fecha_fin.strftime('%d/%m/%Y'))

def _handle_periodo_validation(self, user_id: str, fecha_fin_str: str) -> str:
    """Valida el período completo de vacaciones"""

    user_data = self.state_machine.get_all_data(user_id)
    fecha_inicio = user_data.get('fecha_inicio')
    fecha_fin = datetime.strptime(fecha_fin_str, '%d/%m/%Y').date()

    # 1. Validar rango de fechas
    result_periodo = FechaValidator.validate_periodo(fecha_inicio, fecha_fin)
    if not result_periodo.valid:
        return f"{result_periodo.message}\n 📅 Reingresa los días:"

    # 2. Validar superposición con otras vacaciones
    legajo = user_data.get('legajo')
    result_superposicion = self.context_validator.validate_superposicion(
        legajo, fecha_inicio, user_data.get('dias_solicitados')
    )

    if not result_superposicion['valid']:
        return f"{result_superposicion['message']}\n 📅 Reingresa los días:"

    # 3. Validación contextual completa
    result_contexto = self.context_validator.validate_solicitud_completa(user_data)
    if not result_contexto['valid']:
        # Mostrar todos Los errores encontrados
        mensaje = "❌ Errores encontrados:\n"
        for error in result_contexto['errors']:
            mensaje += f"• {error}\n"
        return mensaje

    # Todo validado: avanzar al gateway de saldo
    self.state_machine.transition_to(user_id, 'verificar_saldo')
    return self._handle_saldo_validation(user_id)

def _handle_saldo_validation(self, user_id: str) -> str:
    """Verifica saldo disponible y toma decisión (Gateway BPMN)"""

```

```

user_data = self.state_machine.get_all_data(user_id)
saldo = user_data.get('empleado', {}).get('saldo_vacaciones', 0)
dias_solicitados = user_data.get('dias_solicitados', 0)

# Gateway XOR: ¿Saldo suficiente?
if dias_solicitados <= saldo:
    self.state_machine.transition_to(user_id, 'aprobacion_jefe')
    return f"✅ Saldo suficiente: {saldo} días disponibles\n" \
           f"📄 Enviando a aprobación de jefe..."
else:
    self.state_machine.transition_to(user_id, 'fin_rechazado')
    return f"❌ Saldo insuficiente. Disponible: {saldo} días\n" \
           f"👉 ¿Deseas ajustar los días solicitados? (si/no)"

def _reset_user_session(self, user_id: str):
    """Reinicia la sesión del usuario después de demasiados errores"""
    self.state_machine.reset_user(user_id)
    self.retry_count[user_id] = 0

```

4. Ejemplos de Interacción (Camino Feliz e Infeliz)

● Camino Feliz

text

👤 : 123456
 🏢 : ✅ Empleado: Juan Pérez
 📅 Ingresar la fecha de inicio (DD/MM/AAAA):

👤 : 15/07/2026
 🏢 : ✅ Fecha: 15/07/2026
 📅 ¿Cuántos días solicitas?

👤 : 10
 🏢 : ✅ Saldo suficiente: 15 días disponibles
 📄 Enviando a aprobación de jefe...

● Camino Infeliz - Legajo

text

👤 : 1234
 🏢 : ❌ El legajo debe tener 6 dígitos (ingresaste 4)
 📅 Intentos restantes: 2

👤 : 123456
 🏢 : ✅ Empleado: Juan Pérez
 📅 Ingresar la fecha de inicio (DD/MM/AAAA):

● Camino Infeliz - Fecha

text

👤 : 32/13/2026
 🏢 : ❌ Formato inválido. Usa DD/MM/AAAA o DD-MM-AAAA
 📅 Intenta nuevamente (DD/MM/AAAA):

👤 : 15/07/2024
 🏢 : ❌ La fecha 15/07/2024 ya pasó
 📅 Intenta nuevamente (DD/MM/AAAA):

 Camino Infeliz - Días

text

 : 35
 :  Máximo 30 días consecutivos (solicitaste 35)
 Reingresa la cantidad de días:

 : 20
 :  Saldo insuficiente. Disponible: 15 días
 ¿Deseas ajustar los días solicitados? (si/no)

5. Documentación para el Diccionario de Datos

markdown

```
## Validaciones Implementadas

| Campo | Regla | Mensaje de Error |
|-----|-----|-----|
| Legajo | - 6 dígitos numéricos<br>- No todo ceros<br>- Debe existir en BD | "El legajo debe tener 6 dígitos" |
| Fecha Inicio | - Formato DD/MM/AAAA<br>- No anterior a hoy<br>- No más de 1 año | "Formato inválido. Usa DD/MM/AAAA" |
| Días | - Número entero positivo<br>- Mínimo 1<br>- Máximo 30<br>- ≤ saldo disponible | "Máximo 30 días consecutivos" |

## Manejo de Errores

1. Reintentos: 3 intentos por campo
2. Recuperación: Posibilidad de corregir datos
3. Validación Contextual: Verifica superposiciones y reglas de negocio
```

Esta implementación garantiza:

- ✓ Validaciones robustas para cada entrada
- ✓ Mensajes de error claros y específicos
- ✓ Sistema de reintentos con límite
- ✓ Recuperación de errores sin perder datos
- ✓ Cobertura del "Camino Infeliz" exigido en el TFI

¿Necesitas que desarrolle algún otro aspecto de las validaciones?

Estoy utilizando OpenPyXL para trabajar con archivos Excel. ¿Cómo puedo buscar empleados, actualizar los días disponibles y registrar nuevas solicitudes de manera segura?

 Gestión de Datos con OpenPyXL para Chatbot de Vacaciones

Te propongo una capa de persistencia robusta usando OpenPyXL que simula una base de datos, cumpliendo con los requisitos del TFI. Incluye manejo de concurrencia, backups y recuperación de errores.

1. Estructura de Archivos Excel

data/empleados.xlsx

legajo	nombre	email	saldo_vacaciones	jefe_email	departamento
123456	Juan Pérez	juan@empresa.com	15	jefe@empresa.com	TI
789012	María Gómez	maria@empresa.com	10	jefe@empresa.com	RRHH

data/solicitudes.xlsx

id_solicitud	legajo	fecha_inicio	fecha_fin	dias	estado	fecha_solicitud
1	123456	2026-07-15	2026-07-24	10	pendiente	2026-06

2. Módulo de Gestión de Datos con OpenPyXL

```
python

# src/data/excel_manager.py

import os
import shutil
import logging
from datetime import datetime, date, timedelta
from typing import Dict, List, Optional, Any, Tuple
from contextlib import contextmanager
from pathlib import Path
import openpyxl
from openpyxl import load_workbook
from openpyxl.utils import get_column_letter
from openpyxl.styles import PatternFill, Font, Alignment
from openpyxl.workbook import Workbook
import threading

# Configuración de Logging
logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)

class ExcelDataManager:
    """
    Gestor de datos con OpenPyXL con protección contra errores
    y manejo de concurrencia
    """

    def __init__(self, data_dir: str = "data", backup_dir: str = "data/backups"):
        self.data_dir = Path(data_dir)
        self.backup_dir = Path(backup_dir)
        self.empleados_file = self.data_dir / "empleados.xlsx"
        self.solicitudes_file = self.data_dir / "solicitudes.xlsx"

        # Crear directorios si no existen
        self.data_dir.mkdir(exist_ok=True)
        self.backup_dir.mkdir(exist_ok=True)

        # Lock para operaciones concurrentes
        self._lock = threading.Lock()

        # Inicializar archivos si no existen
        self._initialize_files()

    def _initialize_files(self):
        """Crea archivos Excel con estructura inicial si no existen"""
```

```

# Archivo de empleados
if not self.empleados_file.exists():
    wb = Workbook()
    ws = wb.active
    ws.title = "Empleados"

# Headers
headers = ['legajo', 'nombre', 'email', 'saldo_vacaciones',
           'jefe_email', 'departamento', 'fecha_ingreso']
for col, header in enumerate(headers, 1):
    ws.cell(row=1, column=col, value=header)
    ws.cell(row=1, column=col).font = Font(bold=True)
    ws.cell(row=1, column=col).fill = PatternFill(
        start_color="CCCCCC", end_color="CCCCCC", fill_type="solid"
    )

# Datos de ejemplo
empleados_ejemplo = [
    ['123456', 'Juan Pérez', 'juan@empresa.com', 15,
     'jefe@empresa.com', 'TI', datetime(2020, 1, 15)],
    ['789012', 'María Gómez', 'maria@empresa.com', 10,
     'jefe@empresa.com', 'RRHH', datetime(2021, 3, 20)],
    ['345678', 'Carlos López', 'carlos@empresa.com', 20,
     'jefe@empresa.com', 'Marketing', datetime(2019, 6, 10)]
]

for row_num, emp_data in enumerate(empleados_ejemplo, 2):
    for col_num, value in enumerate(emp_data, 1):
        ws.cell(row=row_num, column=col_num, value=value)

# Ajustar ancho de columnas
for col in range(1, len(headers) + 1):
    ws.column_dimensions[get_column_letter(col)].width = 20

wb.save(self.empleados_file)
logger.info(f"Archivo empleados.xlsx creado con datos de ejemplo")

# Archivo de solicitudes
if not self.solicitudes_file.exists():
    wb = Workbook()
    ws = wb.active
    ws.title = "Solicitudes"

headers = ['id_solicitud', 'legajo', 'fecha_inicio', 'fecha_fin',
           'dias', 'estado', 'fecha_solicitud', 'motivo',
           'aprobado_por', 'fecha_aprobacion', 'comentarios']
for col, header in enumerate(headers, 1):
    ws.cell(row=1, column=col, value=header)
    ws.cell(row=1, column=col).font = Font(bold=True)
    ws.cell(row=1, column=col).fill = PatternFill(
        start_color="CCCCCC", end_color="CCCCCC", fill_type="solid"
    )

# Ajustar ancho de columnas
for col in range(1, len(headers) + 1):
    ws.column_dimensions[get_column_letter(col)].width = 20

wb.save(self.solicitudes_file)
logger.info(f"Archivo solicitudes.xlsx creado")

@contextmanager
def _safe_workbook(self, file_path: Path, readonly: bool = False):
    """
    Context manager para operaciones seguras con Excel
    Maneja errores y crea backups automáticos
    """
    file_path = Path(file_path)

```

```

try:
    # Crear backup antes de operaciones de escritura
    if not readonly and file_path.exists():
        self._create_backup(file_path)

    wb = load_workbook(file_path, read_only=readonly, data_only=True)
    yield wb

    if not readonly:
        wb.save(file_path)
        wb.close()

except PermissionError as e:
    logger.error(f"Error de permiso: {e}")
    raise PermissionError(f"No se puede acceder al archivo {file_path}")

except Exception as e:
    logger.error(f"Error al procesar archivo {file_path}: {e}")
    # Intentar restaurar backup
    self._restore_backup(file_path)
    raise

def _create_backup(self, file_path: Path):
    """Crea un backup del archivo"""
    timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
    backup_name = f"{file_path.stem}_backup_{timestamp}{file_path.suffix}"
    backup_path = self.backup_dir / backup_name

    try:
        if file_path.exists():
            shutil.copy2(file_path, backup_path)
            # Mantener solo los últimos 10 backups
            self._cleanup_old_backups(file_path.stem)
    except Exception as e:
        logger.warning(f"No se pudo crear backup: {e}")

def _restore_backup(self, file_path: Path):
    """Restaura el último backup disponible"""
    try:
        # Buscar el backup más reciente
        pattern = f"{file_path.stem}_backup_*. {file_path.suffix}"
        backups = sorted(self.backup_dir.glob(pattern))

        if backups:
            latest_backup = backups[-1]
            shutil.copy2(latest_backup, file_path)
            logger.info(f"Restaurado backup: {latest_backup}")
            return True
    except Exception as e:
        logger.error(f"No se pudo restaurar backup: {e}")

    return False

def _cleanup_old_backups(self, file_stem: str, keep: int = 10):
    """Elimina backups antiguos"""
    pattern = f"{file_stem}_backup_*. {self.empleados_file.suffix}"
    backups = sorted(self.backup_dir.glob(pattern))

    for old_backup in backups[:-keep]:
        try:
            old_backup.unlink()
        except Exception as e:
            logger.warning(f"No se pudo eliminar backup antiguo: {e}")

# ===== OPERACIONES DE EMPLEADOS =====

def get_empleado_by_legajo(self, legajo: str) -> Optional[Dict]:

```

```

"""Busca un empleado por número de legajo"""
with self._lock:
    with self._safe_workbook(self.empleados_file, readonly=True) as wb:
        ws = wb.active

        # Buscar por Legajo
        for row in ws.iter_rows(min_row=2, values_only=True):
            if row and str(row[0]) == str(legajo):
                return {
                    'legajo': str(row[0]),
                    'nombre': row[1],
                    'email': row[2],
                    'saldo_vacaciones': row[3],
                    'jefe_email': row[4],
                    'departamento': row[5],
                    'fecha_ingreso': row[6] if isinstance(row[6], date) else None
                }
        return None

def get_empleado_by_email(self, email: str) -> Optional[Dict]:
    """Busca un empleado por email"""
    with self._lock:
        with self._safe_workbook(self.empleados_file, readonly=True) as wb:
            ws = wb.active

            for row in ws.iter_rows(min_row=2, values_only=True):
                if row and str(row[2]) == str(email):
                    return {
                        'legajo': str(row[0]),
                        'nombre': row[1],
                        'email': row[2],
                        'saldo_vacaciones': row[3],
                        'jefe_email': row[4],
                        'departamento': row[5],
                        'fecha_ingreso': row[6] if isinstance(row[6], date) else None
                    }
            return None

def get_all_empleados(self) -> List[Dict]:
    """Obtiene todos los empleados"""
    empleados = []
    with self._lock:
        with self._safe_workbook(self.empleados_file, readonly=True) as wb:
            ws = wb.active

            for row in ws.iter_rows(min_row=2, values_only=True):
                if row and row[0]:
                    empleados.append({
                        'legajo': str(row[0]),
                        'nombre': row[1],
                        'email': row[2],
                        'saldo_vacaciones': row[3],
                        'jefe_email': row[4],
                        'departamento': row[5],
                        'fecha_ingreso': row[6] if isinstance(row[6], date) else None
                    })
            return empleados

def update_saldo_vacaciones(self, legajo: str, dias_a_descontar: int) -> bool:
    """
    Actualiza el saldo de vacaciones de un empleado
    Operación atómica con validación
    """
    with self._lock:
        with self._safe_workbook(self.empleados_file) as wb:
            ws = wb.active

            # Encontrar la fila del empleado

```

```

target_row = None
current_saldo = None

for row_idx, row in enumerate(ws.iter_rows(min_row=2, values_only=True),
start=2):
    if row and str(row[0]) == str(legajo):
        target_row = row_idx
        current_saldo = row[3]
        break

if not target_row:
    logger.warning(f"Empleado {legajo} no encontrado")
    return False

# Validar saldo suficiente
if current_saldo is None or current_saldo < dias_a_descontar:
    logger.warning(f"Saldo insuficiente para {legajo}")
    return False

# Actualizar saldo
nuevo_saldo = current_saldo - dias_a_descontar
ws.cell(row=target_row, column=4, value=nuevo_saldo)

logger.info(f"Saldo actualizado: {legajo} -> {nuevo_saldo} dias")
return True

# ===== OPERACIONES DE SOLICITUDES =====

def registrar_solicitud(self, data: Dict) -> Optional[str]:
    """
    Registra una nueva solicitud de vacaciones
    Retorna el ID de la solicitud
    """
    with self._lock:
        with self._safe_workbook(self.solicitudes_file) as wb:
            ws = wb.active

            # Generar nuevo ID
            next_id = self._get_next_solicitud_id(ws)

            # Obtener última fila
            last_row = ws.max_row + 1

            # Insertar datos
            ws.cell(row=last_row, column=1, value=next_id)
            ws.cell(row=last_row, column=2, value=data.get('legajo'))
            ws.cell(row=last_row, column=3, value=data.get('fecha_inicio'))
            ws.cell(row=last_row, column=4, value=data.get('fecha_fin'))
            ws.cell(row=last_row, column=5, value=data.get('dias'))
            ws.cell(row=last_row, column=6, value=data.get('estado', 'pendiente'))
            ws.cell(row=last_row, column=7, value=datetime.now())
            ws.cell(row=last_row, column=8, value=data.get('motivo', ''))
            ws.cell(row=last_row, column=9, value=data.get('aprobado_por', ''))
            ws.cell(row=last_row, column=10, value=data.get('fecha_aprobacion', ''))
            ws.cell(row=last_row, column=11, value=data.get('comentarios', ''))

            logger.info(f"Solicitud {next_id} registrada para legajo {data.get('legajo')}")

        return str(next_id)

def _get_next_solicitud_id(self, ws) -> int:
    """Obtiene el próximo ID disponible"""
    max_id = 0
    for row in ws.iter_rows(min_row=2, values_only=True):
        if row and row[0]:
            try:
                current_id = int(row[0])
                if current_id > max_id:

```

```

        max_id = current_id
    except (ValueError, TypeError):
        continue
    return max_id + 1

def get_solicitudes_empleado(self, legajo: str, estado: Optional[str] = None) -> List
[Dict]:
    """Obtiene solicitudes de un empleado, opcionalmente filtradas por estado"""
    solicitudes = []
    with self._lock:
        with self._safe_workbook(self.solicitudes_file, readonly=True) as wb:
            ws = wb.active

            for row in ws.iter_rows(min_row=2, values_only=True):
                if row and str(row[1]) == str(legajo):
                    if estado and row[5] != estado:
                        continue

                    solicitudes.append({
                        'id_solicitud': str(row[0]),
                        'legajo': str(row[1]),
                        'fecha_inicio': row[2] if isinstance(row[2], date) else None,
                        'fecha_fin': row[3] if isinstance(row[3], date) else None,
                        'dias': row[4],
                        'estado': row[5],
                        'fecha_solicitud': row[6] if isinstance(row[6], date) else None,
                        'motivo': row[7],
                        'aprobado_por': row[8],
                        'fecha_aprobacion': row[9] if isinstance(row[9], date) else None,
                        'comentarios': row[10]
                    })
    return solicitudes

def actualizar_estado_solicitud(self, id_solicitud: str, nuevo_estado: str,
                                aprobado_por: Optional[str] = None) -> bool:
    """
    Actualiza el estado de una solicitud
    Estados: pendiente, aprobado, rechazado, cancelado
    """
    estados_validos = ['pendiente', 'aprobado', 'rechazado', 'cancelado']
    if nuevo_estado not in estados_validos:
        raise ValueError(f"Estado inválido. Opciones: {' '.join(estados_validos)}")

    with self._lock:
        with self._safe_workbook(self.solicitudes_file) as wb:
            ws = wb.active

            for row_idx, row in enumerate(ws.iter_rows(min_row=2, values_only=True),
start=2):
                if row and str(row[0]) == str(id_solicitud):
                    # Actualizar estado
                    ws.cell(row=row_idx, column=6, value=nuevo_estado)

                    # Si se aprueba, registrar fecha y aprobador
                    if nuevo_estado == 'aprobado':
                        ws.cell(row=row_idx, column=9, value=aprobado_por or 'sistema')

                        ws.cell(row=row_idx, column=10, value=datetime.now())
                    elif nuevo_estado in ['rechazado', 'cancelado']:
                        ws.cell(row=row_idx, column=10, value=datetime.now())

                    logger.info(f"Solicitud {id_solicitud} actualizada a {nuevo_estado}")

                return True

            logger.warning(f"Solicitud {id_solicitud} no encontrada")

```

```

        return False

def get_solicitudes_pendientes_jefe(self, jefe_email: str) -> List[Dict]:
    """Obtiene solicitudes pendientes de aprobación para un jefe"""
    # Primero obtener empleados del jefe
    empleados = self.get_empleados_by_jefe(jefe_email)
    legajos = [emp['legajo'] for emp in empleados]

    solicitudes = []
    with self._lock:
        with self._safe_workbook(self.solicitudes_file, readonly=True) as wb:
            ws = wb.active

            for row in ws.iter_rows(min_row=2, values_only=True):
                if row and str(row[1]) in legajos and row[5] == 'pendiente':
                    solicitudes.append({
                        'id_solicitud': str(row[0]),
                        'legajo': str(row[1]),
                        'fecha_inicio': row[2] if isinstance(row[2], date) else None,
                        'fecha_fin': row[3] if isinstance(row[3], date) else None,
                        'dias': row[4],
                        'estado': row[5],
                        'fecha_solicitud': row[6] if isinstance(row[6], date) else None,
                        'motivo': row[7]
                    })
    return solicitudes

def get_empleados_by_jefe(self, jefe_email: str) -> List[Dict]:
    """Obtiene todos los empleados bajo un jefe"""
    empleados = []
    with self._lock:
        with self._safe_workbook(self.empleados_file, readonly=True) as wb:
            ws = wb.active

            for row in ws.iter_rows(min_row=2, values_only=True):
                if row and str(row[4]) == str(jefe_email):
                    empleados.append({
                        'legajo': str(row[0]),
                        'nombre': row[1],
                        'email': row[2],
                        'saldo_vacaciones': row[3]
                    })
    return empleados

# ===== OPERACIONES DE REPORTERÍA =====

def generar_reporte_solicitudes(self, fecha_inicio: date, fecha_fin: date) -> Dict:
    """Genera un reporte de solicitudes en un período"""
    reporte = {
        'total': 0,
        'aprobadas': 0,
        'rechazadas': 0,
        'pendientes': 0,
        'por_empleado': {}
    }

    with self._lock:
        with self._safe_workbook(self.solicitudes_file, readonly=True) as wb:
            ws = wb.active

            for row in ws.iter_rows(min_row=2, values_only=True):
                if not row or not row[2]:
                    continue

                fecha_solicitud = row[6] if isinstance(row[6], date) else None
                if not fecha_solicitud or not (fecha_inicio <= fecha_solicitud <= fecha_fin):
                    continue

```

```

        continue

    reporte['total'] += 1
    estado = row[5]

    if estado == 'aprobado':
        reporte['aprobadas'] += 1
    elif estado == 'rechazado':
        reporte['rechazadas'] += 1
    elif estado == 'pendiente':
        reporte['pendientes'] += 1

    # Agrupar por empleado
    legajo = str(row[1])
    if legajo not in reporte['por_empleado']:
        reporte['por_empleado'][legajo] = {'total': 0, 'aprobadas': 0}

    reporte['por_empleado'][legajo]['total'] += 1
    if estado == 'aprobado':
        reporte['por_empleado'][legajo]['aprobadas'] += 1

    return reporte

# ===== OPERACIONES DE MANTENIMIENTO =====

def compactar_database(self):
    """Compacta y optimiza los archivos Excel"""
    for file_path in [self.empleados_file, self.solicitudes_file]:
        try:
            with self._safe_workbook(file_path) as wb:
                # Eliminar filas vacías al final
                ws = wb.active
                last_row = ws.max_row
                while last_row > 0:
                    if any(ws.cell(row=last_row, column=col).value
                           for col in range(1, ws.max_column + 1)):
                        break
                    last_row -= 1

                # Si hay filas vacías al final, eliminarlas
                if last_row < ws.max_row:
                    ws.delete_rows(last_row + 1, ws.max_row - last_row)

            logger.info(f"Database compactada: {file_path}")
        except Exception as e:
            logger.error(f"Error compactando {file_path}: {e}")

def validar_integridad(self) -> Dict:
    """Valida la integridad de los datos"""
    resultados = {
        'valido': True,
        'errores': [],
        'advertencias': []
    }

    # Validar empleados duplicados
    legajos = set()
    with self._lock:
        with self._safe_workbook(self.empleados_file, readonly=True) as wb:
            ws = wb.active
            for row in ws.iter_rows(min_row=2, values_only=True):
                if row and row[0]:
                    legajo = str(row[0])
                    if legajo in legajos:
                        resultados['errores'].append(f"Legajo duplicado: {legajo}")
                        resultados['valido'] = False
                    legajos.add(legajo)

```

```
# Validar solicitudes con legajos inexistentes
with self._lock:
    with self._safe_workbook(self.solicitudes_file, readonly=True) as wb:
        ws = wb.active
        for row in ws.iter_rows(min_row=2, values_only=True):
            if row and row[1]:
                legajo = str(row[1])
                if legajo not in legajos:
                    resultados['advertencias'].append(
                        f"Solicitud {row[0]} con legajo inexistente: {legajo}"
                    )

        return resultados
```

3. Servicio de Datos con Transacciones

python

src/data/data_service.py

```
from typing import Dict, Optional, List, Tuple
from datetime import date, datetime, timedelta
from .excel_manager import ExcelDataManager
import logging
```

```
logger = logging.getLogger(__name__)
```

```
class DataService:
```

```
    """
```

```
    Servicio de datos que implementa transacciones y lógica de negocio
    sobre el gestor Excel
    """
```

```
    def __init__(self):
```

```
        self.manager = ExcelDataManager()
```

```
    def procesar_solicitud_vacaciones(self, legajo: str, fecha_inicio: date,
                                       dias: int, motivo: str = "") -> Tuple[bool, str]:
```

```
        """
```

```
        Procesa una solicitud de vacaciones con transacción atómica
        Retorna: (exito, mensaje)
        """
```

```
        try:
```

```
            # 1. Validar empleado
```

```
            empleado = self.manager.get_empleado_by_legajo(legajo)
```

```
            if not empleado:
```

```
                return False, "Empleado no encontrado"
```

```
            # 2. Validar saldo
```

```
            if empleado['saldo_vacaciones'] < dias:
```

```
                return False, f"Saldo insuficiente. Disponible: {empleado['saldo_vacacion
es']} días"
```

```
            # 3. Calcular fecha fin
```

```
            fecha_fin = fecha_inicio + timedelta(days=dias - 1)
```

```
            # 4. Validar superposición con otras vacaciones aprobadas
```

```
            if self._tiene_superposicion(legajo, fecha_inicio, fecha_fin):
```

```
                return False, "Superposición con vacaciones existentes"
```

```
            # 5. Validar días hábiles (opcional)
```

```
            # if not self._son_dias_habiles(fecha_inicio, fecha_fin):
```

```
                # return False, "Las fechas incluyen fines de semana"
```

```
            # 6. Registrar solicitud (sin descontar saldo aún)
```

```
            solicitud_data = {
```

```

        'legajo': legajo,
        'fecha_inicio': fecha_inicio,
        'fecha_fin': fecha_fin,
        'dias': dias,
        'estado': 'pendiente',
        'motivo': motivo
    }

    solicitud_id = self.manager.registrar_solicitud(solicitud_data)

    return True, f"Solicitud {solicitud_id} registrada exitosamente. En espera de
aprobación."

except Exception as e:
    logger.error(f"Error procesando solicitud: {e}")
    return False, f"Error interno: {str(e)}"

def aprobar_solicitud(self, solicitud_id: str, aprobador: str) -> Tuple[bool, str]:
    """
    Aprueba una solicitud y actualiza el saldo
    Operación transaccional
    """
    try:
        # 1. Obtener solicitud
        solicitudes = self.manager.get_solicitudes_empleado('', 'pendiente')
        solicitud = None

        # Buscar por ID
        for s in solicitudes:
            if s['id_solicitud'] == solicitud_id:
                solicitud = s
                break

        if not solicitud:
            return False, "Solicitud no encontrada"

        # 2. Validar que esté pendiente
        if solicitud['estado'] != 'pendiente':
            return False, f"La solicitud ya fue {solicitud['estado']}."

        # 3. Verificar saldo nuevamente (podría haber cambiado)
        empleado = self.manager.get_empleado_by_legajo(solicitud['legajo'])
        if not empleado:
            return False, "Empleado no encontrado"

        if empleado['saldo_vacaciones'] < solicitud['dias']:
            # Rechazar automáticamente por saldo insuficiente
            self.manager.actualizar_estado_solicitud(solicitud_id, 'rechazado', aprobador)

            return False, "Saldo insuficiente. Solicitud rechazada automáticamente."

        # 4. Actualizar estado de la solicitud
        exito = self.manager.actualizar_estado_solicitud(solicitud_id, 'aprobado', aprobador)

        if not exito:
            return False, "Error al actualizar solicitud"

        # 5. Descontar saldo del empleado
        exito_saldo = self.manager.update_saldo_vacaciones(
            solicitud['legajo'],
            solicitud['dias']
        )

        if not exito_saldo:
            # Rollback: revertir estado de solicitud
            self.manager.actualizar_estado_solicitud(solicitud_id, 'pendiente')
            return False, "Error al actualizar saldo. Transacción revertida."

```

```

        return True, f"Solicitud {solicitud_id} aprobada exitosamente"

    except Exception as e:
        logger.error(f"Error aprobando solicitud: {e}")
        return False, f"Error interno: {str(e)}"

def rechazar_solicitud(self, solicitud_id: str, aprobador: str,
                       motivo: str = "") -> Tuple[bool, str]:
    """Rechaza una solicitud sin modificar saldo"""
    try:
        exito = self.manager.actualizar_estado_solicitud(
            solicitud_id, 'rechazado', aprobador
        )

        if not exito:
            return False, "Solicitud no encontrada"

        # Agregar comentarios si hay motivo
        if motivo:
            # Aquí podrías agregar el motivo al campo comentarios
            pass

        return True, f"Solicitud {solicitud_id} rechazada"

    except Exception as e:
        logger.error(f"Error rechazando solicitud: {e}")
        return False, f"Error interno: {str(e)}"

def _tiene_superposicion(self, legajo: str, inicio: date, fin: date) -> bool:
    """Verifica si hay superposición con otras vacaciones aprobadas"""
    solicitudes = self.manager.get_solicitudes_empleado(legajo, 'aprobado')

    for solicitud in solicitudes:
        if not solicitud['fecha_inicio'] or not solicitud['fecha_fin']:
            continue

        # Verificar superposición de rangos
        if not (solicitud['fecha_fin'] < inicio or solicitud['fecha_inicio'] > fin):
            return True

    return False

def _son_dias_habiles(self, inicio: date, fin: date) -> bool:
    """Verifica que todas las fechas sean días hábiles"""
    current = inicio
    while current <= fin:
        if current.weekday() >= 5: # Sábado o Domingo
            return False
        current += timedelta(days=1)
    return True

def obtener_resumen_empleado(self, legajo: str) -> Dict:
    """Obtiene un resumen completo de la situación del empleado"""
    empleado = self.manager.get_empleado_by_legajo(legajo)
    if not empleado:
        return None

    solicitudes = self.manager.get_solicitudes_empleado(legajo)

    # Vacaciones pendientes de aprobación
    pendientes = [s for s in solicitudes if s['estado'] == 'pendiente']

    # Vacaciones aprobadas y no tomadas (futuras)
    aprobadas_futuras = [
        s for s in solicitudes
        if s['estado'] == 'aprobado' and s['fecha_inicio'] > date.today()
    ]

```

```
# Historial de vacaciones tomadas
historial = [
    s for s in solicitudes
    if s['estado'] == 'aprobado' and s['fecha_fin'] <= date.today()
]

return {
    'empleado': empleado,
    'saldo_actual': empleado['saldo_vacaciones'],
    'pendientes': pendientes,
    'aprobadas_futuras': aprobadas_futuras,
    'historial': historial,
    'total_solicitudes': len(solicitudes)
}
```

4. Ejemplo de Uso en el Chatbot

```
python

# src/bot/handlers.py (parte del handler de aprobación)

from src.data.data_service import DataService

class MessageHandler:
    def __init__(self, state_machine):
        self.state_machine = state_machine
        self.data_service = DataService() # Usar DataService en Lugar de ExcelDataManage
r directo

    def _handle_aprobacion_jefe(self, user_id: str, message: str) -> str:
        """Maneja la decisión del jefe (Gateway BPMN)"""

        # Obtener datos de la solicitud
        user_data = self.state_machine.get_all_data(user_id)
        solicitud_id = user_data.get('solicitud_id')
        legajo = user_data.get('legajo')

        if message.lower() in ['si', 'sí', 'aprobar']:
            # Procesar aprobación con transacción
            exito, mensaje = self.data_service.aprobar_solicitud(
                solicitud_id,
                user_data.get('jefe_email', 'jefe')
            )

            if exito:
                self.state_machine.transition_to(user_id, 'fin_aprobado')
                return f"✅ {mensaje}\n📧 Notificación enviada al empleado."
            else:
                return f"❌ Error: {mensaje}"

        elif message.lower() in ['no', 'rechazar']:
            exito, mensaje = self.data_service.rechazar_solicitud(
                solicitud_id,
                user_data.get('jefe_email', 'jefe'),
                "Rechazado por el jefe"
            )

            if exito:
                self.state_machine.transition_to(user_id, 'fin_rechazado')
                return f"❌ {mensaje}"
            else:
                return f"❌ Error: {mensaje}"

        else:
            return "⚠️ Responde 'si' para aprobar o 'no' para rechazar"
```

5. Características de Seguridad Implementadas

Seguridad de Datos

- **Backups automáticos:** Cada modificación crea un backup
- **Restauración automática:** Recuperación ante errores
- **Locks de concurrencia:** Previene corrupción por acceso simultáneo
- **Rollback de transacciones:** Reversión ante fallos

Validaciones

- **Saldo suficiente** antes de aprobar
- **Superposición de fechas** con vacaciones existentes
- **Integridad referencial** entre empleados y solicitudes

Operaciones Atómicas

- Aprobación + descuento de saldo en una transacción
- Rollback automático si falla alguna operación

6. Comandos de Mantenimiento

```
python

# Mantenimiento del sistema (ejecutar periódicamente)

def mantenimiento_diario():
    """Tareas de mantenimiento diario"""
    manager = ExcelDataManager()

    # 1. Compactar archivos
    manager.compactar_database()

    # 2. Validar integridad
    resultados = manager.validar_integridad()

    if not resultados['valido']:
        logger.error(f"Errores de integridad: {resultados}
```

Mi chatbot debe recordar en qué paso del proceso se encuentra el usuario. ¿Cómo puedo implementar una gestión de estados sencilla en Python para representar el flujo BPMN?

Gestión de Estados para Chatbot BPMN

Te propongo una implementación completa de máquina de estados que refleja fielmente el diagrama BPMN, con memoria persistente y manejo de transiciones.

1. Diseño de la Máquina de Estados

Diagrama de Estados (Basado en BPMN)

```
text

[INICIO] → [INGRESO_LEGAJO] → [VALIDAR_LEGAJO] → [INGRESO_FECHAS] → [VALIDAR_FECHAS] →
[VERIFICAR_SALDO] → [GATEWAY_SALDO] → [APROBACION_JEFE] → [GATEWAY_JEFE] → [FIN_APROBADO]
```

2. Implementación Modular de la Máquina de Estados

src/bot/state_machine.py - Núcleo de la Máquina de Estados

```
python

from enum import Enum, auto
from typing import Dict, Any, Optional, List, Callable
from dataclasses import dataclass, field
from datetime import datetime
import json
import logging
from pathlib import Path

logger = logging.getLogger(__name__)

# ===== DEFINICIÓN DE ESTADOS =====

class EstadoBPMN(Enum):
    """Estados del proceso según BPMN"""

    # Estados iniciales
    INICIO = "INICIO"

    # Tareas de usuario (User Tasks)
    INGRESO_LEGajo = "INGRESO_LEGajo"
    INGRESO_FECHAS = "INGRESO_FECHAS"
    INGRESO_DIAS = "INGRESO_DIAS"
    APROBACION_JEFE = "APROBACION_JEFE"

    # Tareas de sistema (Service Tasks)
    VALIDAR_LEGajo = "VALIDAR_LEGajo"
    VALIDAR_FECHAS = "VALIDAR_FECHAS"
    VERIFICAR_SALDO = "VERIFICAR_SALDO"
    CONFIRMAR_SOLICITUD = "CONFIRMAR_SOLICITUD"

    # Gateways (Decisiones)
    GATEWAY_SALDO = "GATEWAY_SALDO"      # XOR: ¿Saldo suficiente?
    GATEWAY_JEFE = "GATEWAY_JEFE"         # XOR: ¿Jefe aprueba?
    GATEWAY_ERROR = "GATEWAY_ERROR"       # Manejo de errores

    # Estados finales (End Events)
    FIN_APROBADO = "FIN_APROBADO"
    FIN_RECHAZADO = "FIN_RECHAZADO"
    FIN_ERROR = "FIN_ERROR"

# ===== CONFIGURACIÓN DE TRANSICIONES =====

@dataclass
class Transicion:
    """Define una transición entre estados"""
    from_state: EstadoBPMN
    to_state: EstadoBPMN
    condition: Optional[Callable] = None
    action: Optional[Callable] = None
    mensaje: str = ""

class MaquinaEstadosBPMN:
    """
    Máquina de estados que implementa el flujo BPMN
    con persistencia y manejo de contexto
    """
```

```

def __init__(self, persistencia_path: str = "data/estados_usuario.json"):
    self.persistencia_path = Path(persistencia_path)
    self.persistencia_path.parent.mkdir(exist_ok=True)

    # Contexto de usuarios: {user_id: EstadoUsuario}
    self.contextos: Dict[str, EstadoUsuario] = {}

    # Configurar transiciones
    self.transiciones = self._configurar_transiciones()

    # Cargar estado persistente
    self._cargar_persistencia()

# ===== CONFIGURACIÓN DE TRANSICIONES =====

def _configurar_transiciones(self) -> Dict[EstadoBPMN, List[Transicion]]:
    """Define el grafo de transiciones según BPMN"""

    transiciones = {
        EstadoBPMN.INICIO: [
            Transicion(
                from_state=EstadoBPMN.INICIO,
                to_state=EstadoBPMN.INGRESO_LEGAJO,
                mensaje="📄 Inicia sesión ingresando tu legajo:"
            )
        ],
        EstadoBPMN.INGRESO_LEGAJO: [
            Transicion(
                from_state=EstadoBPMN.INGRESO_LEGAJO,
                to_state=EstadoBPMN.VALIDAR_LEGAJO,
                condicion=self._validar_legajo,
                mensaje="✅ Validando legajo..."
            )
        ],
        EstadoBPMN.VALIDAR_LEGAJO: [
            Transicion(
                from_state=EstadoBPMN.VALIDAR_LEGAJO,
                to_state=EstadoBPMN.INGRESO_FECHAS,
                condicion=self._legajo_existe,
                mensaje="✅ Empleado encontrado. Ingresar fecha de inicio:"
            ),
            Transicion(
                from_state=EstadoBPMN.VALIDAR_LEGAJO,
                to_state=EstadoBPMN.FIN_RECHAZADO,
                condicion=lambda ctx: not self._legajo_existe(ctx),
                mensaje="❌ Legajo no encontrado en el sistema"
            )
        ],
        EstadoBPMN.INGRESO_FECHAS: [
            Transicion(
                from_state=EstadoBPMN.INGRESO_FECHAS,
                to_state=EstadoBPMN.INGRESO_DIAS,
                condicion=self._validar_fecha,
                mensaje="✅ Fecha válida. ¿Cuántos días solicitás?"
            )
        ],
        EstadoBPMN.INGRESO_DIAS: [
            Transicion(
                from_state=EstadoBPMN.INGRESO_DIAS,
                to_state=EstadoBPMN.VERIFICAR_SALDO,
                condicion=self._validar_dias,
                mensaje="✅ Verificando saldo disponible..."
            )
        ]
    }

```

```

EstadoBPMN.VERIFICAR_SALDO: [
    Transicion(
        from_state=EstadoBPMN.VERIFICAR_SALDO,
        to_state=EstadoBPMN.GATEWAY_SALDO,
        mensaje="🔍 Analizando saldo..."
    )
],

EstadoBPMN.GATEWAY_SALDO: [
    Transicion(
        from_state=EstadoBPMN.GATEWAY_SALDO,
        to_state=EstadoBPMN.APROBACION_JEFE,
        condicion=self._saldo_suficiente,
        mensaje="✅ Saldo suficiente. Enviando a aprobación del jefe..."
    ),
    Transicion(
        from_state=EstadoBPMN.GATEWAY_SALDO,
        to_state=EstadoBPMN.FIN_RECHAZADO,
        condicion=lambda ctx: not self._saldo_suficiente(ctx),
        mensaje="❌ Saldo insuficiente para la solicitud"
    )
],

EstadoBPMN.APROBACION_JEFE: [
    Transicion(
        from_state=EstadoBPMN.APROBACION_JEFE,
        to_state=EstadoBPMN.GATEWAY_JEFE,
        mensaje="📄 Solicitud enviada al jefe. Esperando respuesta..."
    )
],

EstadoBPMN.GATEWAY_JEFE: [
    Transicion(
        from_state=EstadoBPMN.GATEWAY_JEFE,
        to_state=EstadoBPMN.FIN_APROBADO,
        condicion=self._jefe_aprueba,
        mensaje="✅ ¡Solicitud aprobada! Disfruta tus vacaciones 🏖️"
    ),
    Transicion(
        from_state=EstadoBPMN.GATEWAY_JEFE,
        to_state=EstadoBPMN.FIN_RECHAZADO,
        condicion=lambda ctx: not self._jefe_aprueba(ctx),
        mensaje="❌ Solicitud rechazada por el jefe"
    )
],

# Manejo de errores - desde cualquier estado
EstadoBPMN.GATEWAY_ERROR: [
    Transicion(
        from_state=EstadoBPMN.GATEWAY_ERROR,
        to_state=EstadoBPMN.FIN_ERROR,
        mensaje="❌ Error en el proceso. Reinicia el chat."
    )
]
]

return transiciones

# ===== VALIDADORES (Condiciones) =====

def _validar_legajo(self, contexto: 'EstadoUsuario') -> bool:
    """Valida formato del legajo"""
    legajo = contexto.datos.get('legajo', '')
    return legajo.isdigit() and len(legajo) == 6

def _legajo_existe(self, contexto: 'EstadoUsuario') -> bool:
    """Verifica si el legajo existe en la BD"""

```

```

# Esta función se implementa con La capa de datos
legajo = contexto.datos.get('legajo')
# Simulación - en producción usar DataService
return legajo in ['123456', '789012', '345678']

def _validar_fecha(self, contexto: 'EstadoUsuario') -> bool:
    """Valida formato y rango de fecha"""
    fecha = contexto.datos.get('fecha_inicio')
    if not fecha:
        return False
    try:
        # Validar formato DD/MM/AAAA
        datetime.strptime(fecha, '%d/%m/%Y')
        return True
    except ValueError:
        return False

def _validar_dias(self, contexto: 'EstadoUsuario') -> bool:
    """Valida cantidad de días"""
    dias = contexto.datos.get('dias_solicitados')
    try:
        dias_num = int(dias)
        return 1 <= dias_num <= 30
    except (ValueError, TypeError):
        return False

def _saldo_suficiente(self, contexto: 'EstadoUsuario') -> bool:
    """Verifica saldo disponible"""
    saldo = contexto.datos.get('saldo_disponible', 0)
    dias = int(contexto.datos.get('dias_solicitados', 0))
    return dias <= saldo

def _jefe_aprueba(self, contexto: 'EstadoUsuario') -> bool:
    """Simula aprobación del jefe"""
    # En producción, esperaría respuesta del jefe
    # Aquí simulamos con una condición
    return contexto.datos.get('aprobacion_jefe', False)

# ===== GESTIÓN DE ESTADO =====

def obtener_estado(self, user_id: str) -> EstadoBPMN:
    """Obtiene el estado actual de un usuario"""
    if user_id not in self.contextos:
        self.contextos[user_id] = EstadoUsuario(user_id)
    return self.contextos[user_id].estado

def obtener_contexto(self, user_id: str) -> 'EstadoUsuario':
    """Obtiene el contexto completo de un usuario"""
    if user_id not in self.contextos:
        self.contextos[user_id] = EstadoUsuario(user_id)
    return self.contextos[user_id]

def transicionar(self, user_id: str, nuevo_estado: EstadoBPMN,
                 datos: Optional[Dict] = None) -> Dict:
    """
    Realiza una transición de estado
    Retorna: {'exito': bool, 'mensaje': str, 'nuevo_estado': EstadoBPMN}
    """
    contexto = self.obtener_contexto(user_id)
    estado_actual = contexto.estado

    # Buscar transición posible
    transiciones_posibles = self.transiciones.get(estado_actual, [])

    for transicion in transiciones_posibles:
        # Verificar condición si existe
        if transicion.condicion and not transicion.condicion(contexto):
            continue

```

```

        # Ejecutar transición
        contexto.estado = transicion.to_state

        # Actualizar datos
        if datos:
            contexto.datos.update(datos)

        # Ejecutar acción si existe
        if transicion.accion:
            transicion.accion(contexto)

        # Registrar timestamp
        contexto.historial.append({
            'from': estado_actual.value,
            'to': transicion.to_state.value,
            'timestamp': datetime.now().isoformat(),
            'datos': datos
        })

        # Persistir
        self._guardar_persistencia()

        return {
            'exito': True,
            'mensaje': transicion.mensaje,
            'nuevo_estado': transicion.to_state
        }

    # No se encontró transición válida
    logger.warning(f"No se encontró transición de {estado_actual}")
    return {
        'exito': False,
        'mensaje': "⚠️ No se puede avanzar desde este estado",
        'nuevo_estado': estado_actual
    }

def reiniciar(self, user_id: str) -> None:
    """Reinicia el proceso de un usuario"""
    if user_id in self.contextos:
        self.contextos[user_id].reiniciar()
        self._guardar_persistencia()

def es_estado_final(self, estado: EstadoBPMN) -> bool:
    """Verifica si un estado es final"""
    return estado in [
        EstadoBPMN.FIN_APROBADO,
        EstadoBPMN.FIN_RECHAZADO,
        EstadoBPMN.FIN_ERROR
    ]

# ===== PERSISTENCIA =====

def _cargar_persistencia(self):
    """Carga el estado desde archivo"""
    if self.persistencia_path.exists():
        try:
            with open(self.persistencia_path, 'r', encoding='utf-8') as f:
                data = json.load(f)

            for user_id, estado_data in data.items():
                contexto = EstadoUsuario.from_dict(estado_data)
                self.contextos[user_id] = contexto

            logger.info(f"Estado cargado para {len(self.contextos)} usuarios")
        except Exception as e:
            logger.error(f"Error cargando persistencia: {e}")

```

```

def _guardar_persistencia(self):
    """Guarda el estado en archivo"""
    try:
        data = {}
        for user_id, contexto in self.contextos.items():
            data[user_id] = contexto.to_dict()

        with open(self.persistencia_path, 'w', encoding='utf-8') as f:
            json.dump(data, f, indent=2, ensure_ascii=False)
    except Exception as e:
        logger.error(f"Error guardando persistencia: {e}")

# ===== UTILIDADES =====

def listar_estados_activos(self) -> List[str]:
    """Lista usuarios con estados activos"""
    return [user_id for user_id, ctx in self.contextos.items()
            if not self.es_estado_final(ctx.estado)]

def limpiar_estados_antiguos(self, dias: int = 30):
    """Limpia estados inactivos por más de 'dias' días"""
    ahora = datetime.now()
    usuarios_a_eliminar = []

    for user_id, contexto in self.contextos.items():
        if self.es_estado_final(contexto.estado):
            ultima_actualizacion = datetime.fromisoformat(
                contexto.historial[-1]['timestamp'] if contexto.historial else '2020-
01-01'
            )
            if (ahora - ultima_actualizacion).days > dias:
                usuarios_a_eliminar.append(user_id)

    for user_id in usuarios_a_eliminar:
        del self.contextos[user_id]

    if usuarios_a_eliminar:
        self._guardar_persistencia()
        logger.info(f"Eliminados {len(usuarios_a_eliminar)} estados antiguos")

# ===== CONTEXTO DE USUARIO =====

class EstadoUsuario:
    """
    Contiene todo el contexto de un usuario en el proceso
    """

    def __init__(self, user_id: str):
        self.user_id = user_id
        self.estado = EstadoBPMN.INICIO
        self.datos = {} # Diccionario para almacenar datos acumulados
        self.historial = [] # Historial de transiciones
        self.fecha_creacion = datetime.now().isoformat()
        self.ultima_actualizacion = datetime.now().isoformat()
        self.intentos_fallidos = 0
        self.max_intentos = 3

    def reiniciar(self):
        """Reinicia el contexto del usuario"""
        self.estado = EstadoBPMN.INICIO
        self.datos = {}
        self.intentos_fallidos = 0
        self.ultima_actualizacion = datetime.now().isoformat()

    def incrementar_intentos(self) -> bool:
        """Incrementa contador de intentos fallidos"""
        self.intentos_fallidos += 1
        self.ultima_actualizacion = datetime.now().isoformat()

```

```

        return self.intentos_fallidos < self.max_intentos

    def resetear_intentos(self):
        """Reinicia el contador de intentos"""
        self.intentos_fallidos = 0

    def to_dict(self) -> Dict:
        """Serializa a dict para persistencia"""
        return {
            'user_id': self.user_id,
            'estado': self.estado.value,
            'datos': self.datos,
            'historial': self.historial,
            'fecha_creacion': self.fecha_creacion,
            'ultima_actualizacion': self.ultima_actualizacion,
            'intentos_fallidos': self.intentos_fallidos
        }

    @classmethod
    def from_dict(cls, data: Dict) -> 'EstadoUsuario':
        """Deserializa desde dict"""
        contexto = cls(data['user_id'])
        contexto.estado = EstadoBPMN(data['estado'])
        contexto.datos = data.get('datos', {})
        contexto.historial = data.get('historial', [])
        contexto.fecha_creacion = data.get('fecha_creacion', datetime.now().isoformat())
        contexto.ultima_actualizacion = data.get('ultima_actualizacion', datetime.now().isoformat())
        contexto.intentos_fallidos = data.get('intentos_fallidos', 0)
        return contexto

```

3. Integración con el Handler del Chatbot

src/bot/handlers.py - Manejador con Estado

```

python

from src.bot.state_machine import MaquinaEstadosBPMN, EstadoBPMN
from src.data.data_service import DataService
from typing import Dict, Optional
import logging

logger = logging.getLogger(__name__)

class ChatbotHandler:
    """
    Manejador principal del chatbot que utiliza la máquina de estados
    """

    def __init__(self):
        self.maquina_estados = MaquinaEstadosBPMN()
        self.data_service = DataService()

        # Mapeo de estados a funciones manejadoras
        self.manejadores = {
            EstadoBPMN.INICIO: self._manejar_inicio,
            EstadoBPMN.INGRESO_LEGajo: self._manejar_ingreso_legajo,
            EstadoBPMN.VALIDAR_LEGajo: self._manejar_validar_legajo,
            EstadoBPMN.INGRESO_FECHAS: self._manejar_ingreso_fechas,
            EstadoBPMN.INGRESO DIAS: self._manejar_ingreso_dias,
            EstadoBPMN.VERIFICAR_SALDO: self._manejar_verificar_saldo,
            EstadoBPMN.GATEWAY_SALDO: self._manejar_gateway_saldo,
            EstadoBPMN.APROBACION_JEFE: self._manejar_aprobacion_jefe,
            EstadoBPMN.GATEWAY_JEFE: self._manejar_gateway_jefe,
            EstadoBPMN.FIN_APROBADO: self._manejar_fin_aprobado,
            EstadoBPMN.FIN_RECHAZADO: self._manejar_fin_rechazado,

```

```

        EstadoBPMN.FIN_ERROR: self._manejar_fin_error,
    }

def procesar_mensaje(self, user_id: str, mensaje: str) -> str:
    """
    Punto de entrada principal para procesar mensajes
    """
    try:
        # Obtener estado actual
        estado_actual = self.maquina_estados.obtener_estado(user_id)

        # Obtener manejador para el estado actual
        manejador = self.manejadores.get(estado_actual)
        if not manejador:
            return "⚠ Error: Estado no reconocido. Reinicia el proceso."

        # Procesar mensaje según el estado
        respuesta = manejador(user_id, mensaje)

        return respuesta

    except Exception as e:
        logger.error(f"Error procesando mensaje: {e}")
        return "❌ Error inesperado. Por favor, intenta de nuevo."

# ===== MANEJADORES DE ESTADOS =====

def _manejar_inicio(self, user_id: str, mensaje: str) -> str:
    """Estado INICIO: Saludar y pedir legajo"""
    # Transicionar al siguiente estado
    resultado = self.maquina_estados.transicionar(
        user_id,
        EstadoBPMN.INGRESO_LEGajo
    )
    return resultado['mensaje']

def _manejar_ingreso_legajo(self, user_id: str, mensaje: str) -> str:
    """INGRESO_LEGajo: Capturar y validar legajo"""
    contexto = self.maquina_estados.obtener_contexto(user_id)

    # Guardar legajo en contexto
    contexto.datos['legajo'] = mensaje.strip()

    # Validar formato
    if not contexto.datos['legajo'].isdigit() or len(contexto.datos['legajo']) != 6:
        contexto.incrementar_intentos()

        if contexto.intentos_fallidos >= contexto.max_intentos:
            self.maquina_estados.transicionar(user_id, EstadoBPMN.FIN_ERROR)
            return "❌ Demasiados intentos fallidos. Reinicia el proceso."

        return f"❌ Legajo inválido. Debe ser 6 dígitos.\n🔄 Intentos restantes: {contexto.max_intentos - contexto.intentos_fallidos}"

    # Transicionar a validación
    resultado = self.maquina_estados.transicionar(
        user_id,
        EstadoBPMN.VALIDAR_LEGajo
    )

    # Verificar si el legajo existe (se hace en VALIDAR_LEGajo)
    # La transición se ejecuta automáticamente
    return self._manejar_validar_legajo(user_id, mensaje)

def _manejar_validar_legajo(self, user_id: str, mensaje: str) -> str:
    """VALIDAR_LEGajo: Verificar existencia en BD"""
    contexto = self.maquina_estados.obtener_contexto(user_id)
    legajo = contexto.datos.get('legajo')

```

```

# Buscar empleado
empleado = self.data_service.manager.get_empleado_by_legajo(legajo)

if empleado:
    contexto.datos['empleado'] = empleado
    contexto.datos['saldo_disponible'] = empleado['saldo_vacaciones']
    contexto.resetear_intentos()

    # Transicionar a ingreso de fechas
    resultado = self.maquina_estados.transicionar(
        user_id,
        EstadoBPMN.INGRESO_FECHAS
    )
    return f"✅ Empleado: {empleado['nombre']}\n💬 {resultado['mensaje']}"
else:
    # Legajo no existe
    resultado = self.maquina_estados.transicionar(
        user_id,
        EstadoBPMN.FIN_RECHAZADO
    )
    return resultado['mensaje']

def _manejar_ingreso_fechas(self, user_id: str, mensaje: str) -> str:
    """INGRESO_FECHAS: Capturar fecha de inicio"""
    contexto = self.maquina_estados.obtener_contexto(user_id)

    # Validar formato de fecha
    try:
        fecha = datetime.strptime(mensaje.strip(), '%d/%m/%Y').date()
        contexto.datos['fecha_inicio'] = mensaje.strip()
        contexto.resetear_intentos()

        # Transicionar a ingreso de días
        resultado = self.maquina_estados.transicionar(
            user_id,
            EstadoBPMN.INGRESO_DIAS
        )
        return resultado['mensaje']

    except ValueError:
        contexto.incrementar_intentos()

        if contexto.intentos_fallidos >= contexto.max_intentos:
            self.maquina_estados.transicionar(user_id, EstadoBPMN.FIN_ERROR)
            return "❌ Demasiados intentos fallidos. Reinicia el proceso."

        return f"❌ Formato inválido. Usa DD/MM/AAAA\n💬 Intentos restantes: {contexto.max_intentos - contexto.intentos_fallidos}"

def _manejar_ingreso_dias(self, user_id: str, mensaje: str) -> str:
    """INGRESO_DIAS: Capturar cantidad de días"""
    contexto = self.maquina_estados.obtener_contexto(user_id)

    # Validar cantidad de días
    try:
        dias = int(mensaje.strip())
        if dias < 1 or dias > 30:
            return "❌ Los días deben estar entre 1 y 30"

        contexto.datos['dias_solicitados'] = dias
        contexto.resetear_intentos()

        # Transicionar a verificación de saldo
        resultado = self.maquina_estados.transicionar(
            user_id,
            EstadoBPMN.VERIFICAR_SALDO
        )

```

```

        return self._manejar_verificar_saldo(user_id, mensaje)

    except ValueError:
        contexto.incrementar_intentos()

        if contexto.intentos_fallidos >= contexto.max_intentos:
            self.maquina_estados.transicionar(user_id, EstadoBPMN.FIN_ERROR)
            return "❌ Demasiados intentos fallidos. Reinicia el proceso."

        return f"❌ Ingresa un número válido\n🔄 Intentos restantes: {contexto.max_intentos - contexto.intentos_fallidos}"

def _manejar_verificar_saldo(self, user_id: str, mensaje: str) -> str:
    """VERIFICAR_SALDO: Verificar saldo y pasar al gateway"""
    contexto = self.maquina_estados.obtener_contexto(user_id)

    # Transicionar automáticamente al gateway
    resultado = self.maquina_estados.transicionar(
        user_id,
        EstadoBPMN.GATEWAY_SALDO
    )

    return self._manejar_gateway_saldo(user_id, mensaje)

def _manejar_gateway_saldo(self, user_id: str, mensaje: str) -> str:
    """GATEWAY_SALDO: Decisión sobre saldo (XOR)"""
    contexto = self.maquina_estados.obtener_contexto(user_id)

    # La máquina de estados evalúa la condición automáticamente
    # Forzar evaluación de la condición
    saldo = contexto.datos.get('saldo_disponible', 0)
    dias = contexto.datos.get('dias_solicitados', 0)

    if dias <= saldo:
        # Saldo suficiente -> ir a aprobación del jefe
        resultado = self.maquina_estados.transicionar(
            user_id,
            EstadoBPMN.APROBACION_JEFE
        )
        return f"✅ Saldo disponible: {saldo} días\n{resultado['mensaje']}"
    else:
        # Saldo insuficiente -> fin rechazado
        resultado = self.maquina_estados.transicionar(
            user_id,
            EstadoBPMN.FIN_RECHAZADO
        )
        return f"❌ Saldo insuficiente\nDisponible: {saldo} días\nSolicitados: {dias} días\n{resultado['mensaje']}"

def _manejar_aprobacion_jefe(self, user_id: str, mensaje: str) -> str:
    """APROBACION_JEFE: Simular envío al jefe"""
    contexto = self.maquina_estados.obtener_contexto(user_id)

    # En producción aquí se enviaría la solicitud al jefe
    # Simulamos respuesta
    contexto.datos['aprobacion_jefe'] = True # Simular aprobación

    # Transicionar al gateway del jefe
    resultado = self.maquina_estados.transicionar(
        user_id,
        EstadoBPMN.GATEWAY_JEFE
    )

    return self._manejar_gateway_jefe(user_id, mensaje)

def _manejar_gateway_jefe(self, user_id: str, mensaje: str) -> str:
    """GATEWAY_JEFE: Decisión del jefe (XOR)"""
    contexto = self.maquina_estados.obtener_contexto(user_id)

```

```

# Evaluar decisión del jefe
if contexto.datos.get('aprobacion_jefe', False):
    resultado = self.maquina_estados.transicionar(
        user_id,
        EstadoBPMN.FIN_APROBADO
    )
    return resultado['mensaje']
else:
    resultado = self.maquina_estados.transicionar(
        user_id,
        EstadoBPMN.FIN_RECHAZADO
    )
    return resultado['mensaje']

def _manejar_fin_aprobado(self, user_id: str, mensaje: str) -> str:
    """FIN_APROBADO: Estado final exitoso"""
    return "✅ Proceso completado exitosamente. Escribe 'inicio' para comenzar de nuevo."

def _manejar_fin_rechazado(self, user_id: str, mensaje: str) -> str:
    """FIN_RECHAZADO: Estado final rechazado"""
    return "❌ Proceso rechazado. Escribe 'inicio' para intentar de nuevo."

def _manejar_fin_error(self, user_id: str, mensaje: str) -> str:
    """FIN_ERROR: Estado final por error"""
    self.maquina_estados.reiniciar(user_id)
    return "❌ Error en el proceso. Reiniciado. Escribe 'inicio' para comenzar."

```

4. Integración con el Handler del Chatbot

src/main.py - Punto de Entrada

```

python

from src.bot.handlers import ChatbotHandler
import sys

def main():
    """Simulación de chatbot en consola"""
    handler = ChatbotHandler()

    print("=" * 50)
    print("🤖 CHATBOT DE GESTIÓN DE VACACIONES")
    print("=" * 50)
    print("Comandos especiales:")
    print(" • 'estado' - Ver estado actual")
    print(" • 'reiniciar' - Reiniciar proceso")
    print(" • 'salir' - Salir del sistema")
    print("=" * 50)

    user_id = "usuario_test"

    # Mensaje de bienvenida
    respuesta = handler.procesar_mensaje(user_id, "inicio")
    print(f"\n🤖: {respuesta}")

    while True:
        try:
            # Leer entrada del usuario
            user_input = input("\n👤 Tú: ").strip()

            if user_input.lower() == 'salir':
                print("👋 ¡Hasta luego!")
                break

```

```

if user_input.lower() == 'reiniciar':
    handler.maquina_estados.reiniciar(user_id)
    respuesta = "🔄 Proceso reiniciado. Escribe 'inicio' para comenzar."
    print(f"\n🤖: {respuesta}")
    continue

if user_input.lower() == 'estado':
    contexto = handler.maquina_estados.obtener_contexto(user_id)
    print(f"\n📋 Estado actual: {contexto.estado.value}")
    print(f"\n📁 Datos: {contexto.datos}")
    continue

# Procesar mensaje
respuesta = handler.procesar_mensaje(user_id, user_input)
print(f"\n🤖: {respuesta}")

# Verificar si llegamos a un estado final
estado_actual = handler.maquina_estados.obtener_estado(user_id)
if handler.maquina_estados.es_estado_final(estado_actual):
    print("\n🏁 Proceso finalizado. Escribe 'reiniciar' para comenzar de nuev
o.")

except KeyboardInterrupt:
    print("\n👋 ¡Hasta luego!")
    break
except Exception as e:
    print(f"\n❌ Error: {e}")

if __name__ == "__main__":
    main()

```

5. Persistencia de Estado en JSON

Ejemplo de Archivo de Persistencia (data/estados_usuario.json)

```

json

{
  "usuario_test": {
    "user_id": "usuario_test",
    "estado": "APROBACION_JEFE",
    "datos": {
      "legajo": "123456",
      "empleado": {
        "legajo": "123456",
        "nombre": "Juan Pérez",
        "email": "juan@empresa.com",
        "saldo_vacaciones": 15,
        "jefe_email": "jefe@empresa.com",
        "departamento": "TI"
      },
      "saldo_disponible": 15,
      "fecha_inicio": "15/07/2026",
      "dias_solicitados": 10
    },
    "historial": [
      {
        "from": "INICIO",
        "to": "INGRESO_LEGAJO",
        "timestamp": "2026-06-19T10:30:00",
        "datos": null
      },
      {
        "from": "INGRESO_LEGAJO",
        "to": "VALIDAR_LEGAJO",
        "timestamp": "2026-06-19T10:30:15",

```

```
        "datos": {
            "legajo": "123456"
        }
    },
    "fecha_creacion": "2026-06-19T10:30:00",
    "ultima_actualizacion": "2026-06-19T10:35:00",
    "intentos_fallidos": 0
}
```

6. Ventajas de esta Implementación

✓ Cumple con los Requisitos del TFI

Requisito	Implementación
Máquina de Estados	MaquinaEstadosBPMN con transiciones y condiciones
Memoria de usuario	Contexto persistente en JSON
Gateway BPMN	GATEWAY_SALDO y GATEWAY_JEFE con condiciones XOR
Camino Feliz/Infeliz	Manejo de errores y reintentos
Persistencia	

 **main.py**
PY 17.9KB

Te comparto mi código Python. ¿Podés revisarlo y sugerirme mejoras relacionadas con validaciones, organización del código, claridad y experiencia del usuario?

Revisión y Mejoras del Código del Chatbot de Vacaciones

¡Excelente trabajo! Tu código es funcional y bien estructurado. Te propongo mejoras en validaciones, organización, claridad y experiencia de usuario, manteniendo la esencia de tu implementación.

1. Análisis y Mejoras Propuestas

📊 Resumen de Mejoras

Aspecto	Estado Actual	Mejora Propuesta
Validaciones	Básicas	Validaciones más robustas y específicas
Organización	Funcional	Separación en módulos según responsabilidad
Manejo de Errores	Básico	Sistema de reintentos y recuperación
Experiencia de Usuario	Buena	Mensajes más claros y guiados
Persistencia	Simple	Backups y manejo de concurrencia
Código	Limpio	Más modular y reutilizable

2. Código Mejorado y Modularizado

Estructura Propuesta

```

text

chatbot_vacaciones/
├── src/
│   ├── __init__.py
│   ├── main.py                # Punto de entrada
│   ├── config/
│   │   └── settings.py        # Configuraciones
│   ├── data/
│   │   ├── __init__.py
│   │   └── excel_manager.py    # Gestión de Excel mejorada
│   ├── validators/
│   │   ├── __init__.py
│   │   └── validators.py       # Validaciones separadas
│   ├── bot/
│   │   ├── __init__.py
│   │   ├── handlers.py        # Lógica de conversación
│   │   └── state_machine.py    # Gestión de estados
│   └── utils/
│       ├── __init__.py
│       └── logger.py           # Logging
├── base_datos/
│   ├── empleados.xlsx
│   └── solicitudes.xlsx
├── tests/
│   └── test_validators.py
├── requirements.txt
└── README.md

```

3. Módulo de Validaciones Mejorado

src/validators/validators.py

```

python

"""
Módulo de validaciones para el chatbot de vacaciones
Incluye validaciones robustas con mensajes claros y sistema de reintentos
"""

from datetime import datetime, date
from typing import Tuple, Optional, Dict, Any
import re

class ValidationResult:
    """
    Estandariza los resultados de validación
    """
    def __init__(self, valid: bool, data: Any = None,
                  error_code: str = None, message: str = None):
        self.valid = valid
        self.data = data
        self.error_code = error_code
        self.message = message or self._get_default_message(error_code)

    def _get_default_message(self, code: str) -> str:

```

```

"""Mensajes predefinidos por tipo de error"""
messages = {
    'EMPTY': '⚠ El campo no puede estar vacío.',
    'INVALID_FORMAT': '❌ Formato incorrecto.',
    'INVALID_LENGTH': '❌ Longitud incorrecta.',
    'INVALID_CHARS': '❌ Caracteres no permitidos.',
    'PAST_DATE': '❌ La fecha no puede ser anterior a hoy.',
    'INSUFFICIENT_DAYS': '❌ Días insuficientes.',
    'MAX_EXCEEDED': '❌ Supera el máximo permitido.',
    'ZERO_DAYS': '❌ La cantidad debe ser mayor a 0.',
    'NOT_FOUND': '❌ No se encontró en el sistema.'
}

return messages.get(code, '❌ Dato inválido.')

class LegajoValidator:
    """Validador para números de legajo"""

    @staticmethod
    def validate(legajo: str) -> ValidationResult:
        """
        Valida formato del legajo (solo números, 6 dígitos)
        """
        # 1. Limpiar entrada
        legajo = legajo.strip() if legajo else ""

        # 2. Validar vacío
        if not legajo:
            return ValidationResult(False, error_code='EMPTY')

        # 3. Validar solo números
        if not legajo.isdigit():
            return ValidationResult(
                False,
                error_code='INVALID_CHARS',
                message='❌ El legajo debe contener solo números.'
            )

        # 4. Validar Longitud (6 dígitos)
        if len(legajo) != 6:
            return ValidationResult(
                False,
                error_code='INVALID_LENGTH',
                message=f'❌ El legajo debe tener 6 dígitos (ingresaste {len(legajo)}).'
            )

        # 5. Validación de negocio: no puede ser todo ceros
        if legajo == '000000':
            return ValidationResult(
                False,
                error_code='INVALID_FORMAT',
                message='❌ Legajo inválido.'
            )

        return ValidationResult(True, data={'legajo': legajo})

class FechaValidator:
    """Validador para fechas"""

    @staticmethod
    def validate(fecha_str: str, permitir_pasado: bool = False) -> ValidationResult:
        """
        Valida formato DD/MM/AAAA y fecha futura

        Args:
            fecha_str: Fecha en formato DD/MM/AAAA
            permitir_pasado: Si permite fechas pasadas (para pruebas)

```

```

"""
# 1. Limpiar entrada
fecha_str = fecha_str.strip() if fecha_str else ""

# 2. Validar vacío
if not fecha_str:
    return ValidationResult(False, error_code='EMPTY')

# 3. Validar formato
try:
    fecha = datetime.strptime(fecha_str, '%d/%m/%Y').date()
except ValueError:
    # Intentar con formato alternativo
    try:
        fecha = datetime.strptime(fecha_str, '%d-%m-%Y').date()
        fecha_str = fecha.strftime('%d/%m/%Y')
    except ValueError:
        return ValidationResult(
            False,
            error_code='INVALID_FORMAT',
            message='❌ Formato inválido. Usa DD/MM/AAAA o DD-MM-AAAA.'
        )

# 4. Validar fecha futura
if not permitir_pasado:
    hoy = date.today()
    if fecha < hoy:
        return ValidationResult(
            False,
            error_code='PAST_DATE',
            message=f'❌ La fecha {fecha.strftime("%d/%m/%Y")} ya pasó.'
        )

# 5. Validar rango (no más de 1 año en el futuro)
max_fecha = date.today().replace(year=date.today().year + 1)
if fecha > max_fecha:
    return ValidationResult(
        False,
        error_code='INVALID_RANGE',
        message='❌ Solo se pueden solicitar vacaciones con hasta 1 año de anticipación.'
    )

# 6. Validar día hábil (opcional - con advertencia)
if fecha.weekday() in [5, 6]: # Sábado o Domingo
    return ValidationResult(
        False,
        error_code='WEEKEND',
        message=f'⚠️ {fecha.strftime("%A")} es fin de semana. ¿Estás seguro?'
    )

return ValidationResult(True, data={'fecha': fecha, 'fecha_str': fecha_str})

class DiasValidator:
    """Validador para cantidad de días"""

    MIN_DIAS = 1
    MAX_DIAS = 30

    @classmethod
    def validate(cls, dias_str: str, saldo_disponible: Optional[int] = None) -> ValidationResult:
        """
        Valida cantidad de días solicitados

        Args:
            dias_str: String con la cantidad de días

```

```

        saldo_disponible: Saldo disponible para validar contra él
    """

    # 1. Limpiar entrada
    dias_str = dias_str.strip() if dias_str else ""

    # 2. Validar vacío
    if not dias_str:
        return ValidationResult(False, error_code='EMPTY')

    # 3. Validar solo números
    if not dias_str.isdigit():
        return ValidationResult(
            False,
            error_code='INVALID_CHARS',
            message='❌ Debes ingresar un número entero.'
        )

    # 4. Convertir a entero
    dias = int(dias_str)

    # 5. Validar mínimo
    if dias < cls.MIN_DIAS:
        return ValidationResult(
            False,
            error_code='ZERO_DAYS',
            message=f'❌ Mínimo {cls.MIN_DIAS} día de vacaciones.'
        )

    # 6. Validar máximo
    if dias > cls.MAX_DIAS:
        return ValidationResult(
            False,
            error_code='MAX_EXCEEDED',
            message=f'❌ Máximo {cls.MAX_DIAS} días consecutivos (solicitaste {dias}).'
        )

    # 7. Validar contra saldo disponible
    if saldo_disponible is not None:
        if dias > saldo_disponible:
            return ValidationResult(
                False,
                error_code='INSUFFICIENT_DAYS',
                message=f'❌ Saldo insuficiente. Disponible: {saldo_disponible} días.'
            )

    return ValidationResult(True, data={'dias': dias})

class SolicitudValidator:
    """Validador contextual para la solicitud completa"""

    @staticmethod
    def validate_solicitud(datos: Dict) -> Dict:
        """
        Valida todos los datos de la solicitud juntos
        Retorna: {'valid': bool, 'errors': List[str], 'warnings': List[str]}
        """
        result = {
            'valid': True,
            'errors': [],
            'warnings': []
        }

        # Validar legajo
        if 'legajo' in datos:
            val = LegajoValidator.validate(datos['legajo'])

```

```

        if not val.valid:
            result['valid'] = False
            result['errors'].append(f"Legajo: {val.message}")

    # Validar fecha
    if 'fecha_inicio' in datos:
        val = FechaValidator.validate(datos['fecha_inicio'])
        if not val.valid:
            result['valid'] = False
            result['errors'].append(f"Fecha: {val.message}")

    # Validar días
    if 'dias_solicitados' in datos:
        saldo = datos.get('saldo_disponible')
        val = DiasValidator.validate(str(datos['dias_solicitados']), saldo)
        if not val.valid:
            result['valid'] = False
            result['errors'].append(f"Días: {val.message}")

    return result

```

4. Gestor de Excel Mejorado

src/data/excel_manager.py

```

python

"""
Gestor de archivos Excel con manejo de errores, backups y transacciones
"""

import os
import shutil
from pathlib import Path
from datetime import datetime
from typing import Optional, Dict, Any, List
from contextlib import contextmanager
import threading
import logging

from openpyxl import load_workbook, Workbook
from openpyxl.utils import get_column_letter
from openpyxl.styles import PatternFill, Font, Alignment

logger = logging.getLogger(__name__)

class ExcelManager:
    """
    Gestor de archivos Excel con protección contra errores
    y manejo de concurrencia
    """

    # Mapeo de columnas por archivo
    COLUMNAS_EMPLEADOS = {
        'legajo': 1,
        'nombre': 2,
        'sector': 3,
        'dias_disponibles': 4
    }

    COLUMNAS_SOLICITUDES = {
        'id': 1,
        'legajo': 2,
        'nombre': 3,

```

```

        'sector': 4,
        'fecha_inicio': 5,
        'dias': 6,
        'estado': 7
    }

def __init__(self, data_dir: str = "base_datos"):
    self.data_dir = Path(data_dir)
    self.backup_dir = self.data_dir / "backups"

    # Crear directorios
    self.data_dir.mkdir(exist_ok=True)
    self.backup_dir.mkdir(exist_ok=True)

    # Archivos
    self.empleados_file = self.data_dir / "empleados.xlsx"
    self.solicitudes_file = self.data_dir / "solicitudes.xlsx"

    # Lock para concurrencia
    self._lock = threading.Lock()

    # Inicializar archivos
    self._initialize_files()

def _initialize_files(self):
    """Crea archivos con estructura si no existen"""
    # Archivo de empleados
    if not self.empleados_file.exists():
        self._create_empleados_template()

    # Archivo de solicitudes
    if not self.solicitudes_file.exists():
        self._create_solicitudes_template()

def _create_empleados_template(self):
    """Crea template de empleados con datos de ejemplo"""
    wb = Workbook()
    ws = wb.active
    ws.title = "Empleados"

    # Headers
    headers = ['Legajo', 'Nombre', 'Sector', 'Días Disponibles']
    for col, header in enumerate(headers, 1):
        ws.cell(row=1, column=col, value=header)
        ws.cell(row=1, column=col).font = Font(bold=True)
        ws.cell(row=1, column=col).fill = PatternFill(
            start_color="CCCCCC", end_color="CCCCCC", fill_type="solid"
        )

    # Datos de ejemplo
    empleados = [
        ['123456', 'Juan Pérez', 'TI', 15],
        ['789012', 'María Gómez', 'RRHH', 10],
        ['345678', 'Carlos López', 'Marketing', 20],
        ['901234', 'Ana Martínez', 'Finanzas', 12],
        ['567890', 'Pedro Rodríguez', 'Operaciones', 8]
    ]

    for row, data in enumerate(empleados, 2):
        for col, value in enumerate(data, 1):
            ws.cell(row=row, column=col, value=value)

    # Ajustar columnas
    for col in range(1, 5):
        ws.column_dimensions[get_column_letter(col)].width = 20

    wb.save(self.empleados_file)
    logger.info("Archivo empleados.xlsx creado con datos de ejemplo")

```

```

def _create_solicitudes_template(self):
    """Crea template de solicitudes"""
    wb = Workbook()
    ws = wb.active
    ws.title = "Solicitudes"

    # Headers
    headers = ['ID', 'Legajo', 'Nombre', 'Sector', 'Fecha Inicio', 'Días', 'Estado']
    for col, header in enumerate(headers, 1):
        ws.cell(row=1, column=col, value=header)
        ws.cell(row=1, column=col).font = Font(bold=True)
        ws.cell(row=1, column=col).fill = PatternFill(
            start_color="CCCCCC", end_color="CCCCCC", fill_type="solid"
        )

    # Ajustar columnas
    for col in range(1, 8):
        ws.column_dimensions[get_column_letter(col)].width = 20

    wb.save(self.solicitudes_file)
    logger.info("Archivo solicitudes.xlsx creado")

@contextmanager
def _safe_workbook(self, file_path: Path, readonly: bool = False):
    """
    Context manager para operaciones seguras con Excel
    """
    file_path = Path(file_path)

    try:
        # Backup antes de escritura
        if not readonly and file_path.exists():
            self._create_backup(file_path)

        wb = load_workbook(file_path, read_only=readonly, data_only=True)
        yield wb

        if not readonly:
            wb.save(file_path)
            wb.close()

    except Exception as e:
        logger.error(f"Error en operación con {file_path}: {e}")
        # Intentar restaurar backup
        self._restore_backup(file_path)
        raise

def _create_backup(self, file_path: Path):
    """Crea backup del archivo"""
    timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
    backup_name = f"{file_path.stem}_backup_{timestamp}{file_path.suffix}"
    backup_path = self.backup_dir / backup_name

    try:
        shutil.copy2(file_path, backup_path)
        # Limpiar backups antiguos
        self._cleanup_old_backups(file_path.stem)
    except Exception as e:
        logger.warning(f"No se pudo crear backup: {e}")

def _restore_backup(self, file_path: Path) -> bool:
    """Restaura el último backup"""
    try:
        pattern = f"{file_path.stem}_backup_*. {file_path.suffix}"
        backups = sorted(self.backup_dir.glob(pattern))

        if backups:

```

```

        shutil.copy2(backups[-1], file_path)
        logger.info(f"Backup restaurado: {backups[-1]}")
        return True
    except Exception as e:
        logger.error(f"Error restaurando backup: {e}")
    return False

def _cleanup_old_backups(self, file_stem: str, keep: int = 5):
    """Elimina backups antiguos"""
    pattern = f"{file_stem}_backup_{self.empleados_file.suffix}"
    backups = sorted(self.backup_dir.glob(pattern))

    for old in backups[:-keep]:
        try:
            old.unlink()
        except Exception:
            pass

# ===== OPERACIONES DE EMPLEADOS =====

def buscar_empleado(self, legajo: str) -> Optional[Dict]:
    """
    Busca un empleado por legajo
    Retorna dict con datos o None
    """
    with self._lock:
        with self._safe_workbook(self.empleados_file, readonly=True) as wb:
            ws = wb.active

            for row in ws.iter_rows(min_row=2, values_only=True):
                if row and row[0] and str(row[0]) == legajo:
                    return {
                        'fila': row[0], # Usamos el valor del legajo como identifica
                        'legajo': str(row[0]),
                        'nombre': row[1],
                        'sector': row[2],
                        'dias': int(row[3]) if row[3] else 0
                    }
            return None

def obtener_datos_empleado_completo(self, legajo: str) -> Optional[Dict]:
    """
    Obtiene datos completos del empleado incluyendo referencias para actualizar
    """
    with self._lock:
        # Abrir en modo normal para poder actualizar después
        wb = load_workbook(self.empleados_file)
        ws = wb.active

        for row in ws.iter_rows(min_row=2):
            if row and row[0].value and str(row[0].value) == legajo:
                return {
                    'fila': row[0].row,
                    'legajo': str(row[0].value),
                    'nombre': row[1].value,
                    'sector': row[2].value,
                    'dias': int(row[3].value) if row[3].value else 0,
                    'workbook': wb,
                    'hoja': ws
                }

        wb.close()
        return None

def actualizar_dias_empleado(self, legajo: str, nuevos_dias: int) -> bool:
    """
    Actualiza los días disponibles de un empleado

```

```

"""
with self._lock:
    with self._safe_workbook(self.empleados_file) as wb:
        ws = wb.active

        for row in ws.iter_rows(min_row=2):
            if row and row[0].value and str(row[0].value) == legajo:
                row[3].value = nuevos_dias
                logger.info(f"Días actualizados para {legajo}: {nuevos_dias}")
                return True

        logger.warning(f"Empleado {legajo} no encontrado para actualizar")
        return False

def get_saldo_empleado(self, legajo: str) -> Optional[int]:
    """Obtiene el saldo de días de un empleado"""
    empleado = self.buscar_empleado(legajo)
    return empleado['dias'] if empleado else None

# ===== OPERACIONES DE SOLICITUDES =====

def registrar_solicitud(self, legajo: str, nombre: str, sector: str,
                        fecha_inicio: str, dias: int, estado: str) -> int:
    """
    Registra una nueva solicitud en el historial
    Retorna el ID de la solicitud
    """
    with self._lock:
        with self._safe_workbook(self.solicitudes_file) as wb:
            ws = wb.active

            # Generar ID
            nuevo_id = ws.max_row # El ID será el número de fila

            # Agregar fila
            ws.cell(row=nuevo_id + 1, column=1, value=nuevo_id)
            ws.cell(row=nuevo_id + 1, column=2, value=legajo)
            ws.cell(row=nuevo_id + 1, column=3, value=nombre)
            ws.cell(row=nuevo_id + 1, column=4, value=sector)
            ws.cell(row=nuevo_id + 1, column=5, value=fecha_inicio)
            ws.cell(row=nuevo_id + 1, column=6, value=dias)
            ws.cell(row=nuevo_id + 1, column=7, value=estado)

            logger.info(f"Solicitud {nuevo_id} registrada para {legajo}")
            return nuevo_id

def obtener_solicitudes_empleado(self, legajo: str) -> List[Dict]:
    """Obtiene todas las solicitudes de un empleado"""
    solicitudes = []

    with self._lock:
        with self._safe_workbook(self.solicitudes_file, readonly=True) as wb:
            ws = wb.active

            for row in ws.iter_rows(min_row=2, values_only=True):
                if row and row[1] and str(row[1]) == legajo:
                    solicitudes.append({
                        'id': row[0],
                        'legajo': str(row[1]),
                        'nombre': row[2],
                        'sector': row[3],
                        'fecha_inicio': row[4],
                        'dias': row[5],
                        'estado': row[6]
                    })

    return solicitudes

```

```

def actualizar_estado_solicitud(self, solicitud_id: int, nuevo_estado: str) -> bool:
    """Actualiza el estado de una solicitud"""
    with self._lock:
        with self._safe_workbook(self.solicitudes_file) as wb:
            ws = wb.active

            # Buscar por ID (que es el número de fila)
            target_row = solicitud_id + 1 # +1 por el header

            if target_row <= ws.max_row:
                ws.cell(row=target_row, column=7, value=nuevo_estado)
                logger.info(f"Solicitud {solicitud_id} actualizada a {nuevo_estado}")
                return True

            logger.warning(f"Solicitud {solicitud_id} no encontrada")
            return False

# ===== OPERACIONES DE MANTENIMIENTO =====

def validar_integridad(self) -> Dict:
    """Valida la integridad de la base de datos"""
    resultado = {'valido': True, 'errores': [], 'advertencias': []}

    # Verificar archivos
    for file_path in [self.empleados_file, self.solicitudes_file]:
        if not file_path.exists():
            resultado['errores'].append(f"Archivo no encontrado: {file_path}")
            resultado['valido'] = False

    # Verificar que el archivo de empleados tenga datos
    if self.empleados_file.exists():
        empleados = self.buscar_todos_empleados()
        if not empleados:
            resultado['advertencias'].append("No hay empleados cargados")

    return resultado

def buscar_todos_empleados(self) -> List[Dict]:
    """Obtiene todos los empleados"""
    empleados = []

    with self._lock:
        with self._safe_workbook(self.empleados_file, readonly=True) as wb:
            ws = wb.active

            for row in ws.iter_rows(min_row=2, values_only=True):
                if row and row[0]:
                    empleados.append({
                        'legajo': str(row[0]),
                        'nombre': row[1],
                        'sector': row[2],
                        'dias': int(row[3]) if row[3] else 0
                    })

    return empleados

```

5. Handler del Chatbot Mejorado

src/bot/handlers.py

python

"""

Manejador del chatbot con gestión de estados y validaciones robustas

```

"""

import sys
from typing import Optional, Dict, Any
from datetime import datetime
import logging

from src.validators.validators import (
    LegajoValidator, FechaValidator, DiasValidator,
    SolicitudValidator, ValidationResult
)
from src.data.excel_manager import ExcelManager

logger = logging.getLogger(__name__)

class ChatbotHandler:
    """
    Manejador principal del chatbot con sistema de estados
    """

    # Estados del proceso
    ESTADO_INICIO = "INICIO"
    ESTADO_SOLICITANDO_LEGajo = "SOLICITANDO_LEGajo"
    ESTADO_SOLICITANDO_FECHA = "SOLICITANDO_FECHA"
    ESTADO_SOLICITANDO_DIAS = "SOLICITANDO_DIAS"
    ESTADO_EVALUANDO = "EVALUANDO"
    ESTADO_APROBACION_SUPERVISOR = "APROBACION_SUPERVISOR"
    ESTADO_FINALIZADO = "FINALIZADO"

    # Configuración
    MAX_INTENTOS = 3
    DIAS_APROBACION_AUTOMATICA = 10

    def __init__(self):
        self.excel_manager = ExcelManager()
        self.estado_actual = self.ESTADO_INICIO
        self.datos_solicitud = {}
        self.intentos = {}

    def iniciar(self):
        """Inicia el flujo del chatbot"""
        self._mostrar_bienvenida()
        self._procesar_legajo()

    # ===== MÉTODOS PRIVADOS =====

    def _mostrar_bienvenida(self):
        """Muestra mensaje de bienvenida"""
        print("=" * 60)
        print("🤖 CHATBOT DE GESTIÓN DE VACACIONES")
        print("📁 BCH SISTEMAS")
        print("=" * 60)
        print("\n👋 ¡Bienvenido al sistema de gestión de vacaciones!")
        print("👤 Este asistente te ayudará a registrar tu solicitud.")
        print("💡 En cualquier momento, escribe 'salir' para finalizar.")
        print("=" * 60)

    def _procesar_legajo(self):
        """Procesa el ingreso del legajo con validaciones"""
        self.estado_actual = self.ESTADO_SOLICITANDO_LEGajo

        while True:
            print("\n📄 Ingresa tu número de legajo:")
            legajo_input = input("👉 Legajo: ").strip()

            if self._verificar_salida(legajo_input):
                return

```

```

# Validar Legajo
resultado = LegajoValidator.validate(legajo_input)

if not resultado.valid:
    self._incrementar_intentos('legajo')
    print(f"\n🚫 {resultado.message}")

    if self.intentos.get('legajo', 0) >= self.MAX_INTENTOS:
        print("\n❌ Demasiados intentos fallidos. El proceso ha sido cancelad
o.")

        sys.exit(0)
        continue

# Buscar empleado en la base de datos
legajo = resultado.data['legajo']
empleado = self.excel_manager.buscar_empleado(legajo)

if not empleado:
    self._incrementar_intentos('legajo')
    print("\n❌ No se encontró un empleado con ese legajo.")
    print("💡 Verifica que el número sea correcto.")
    continue

# Empleado encontrado
self.datos_solicitud['empleado'] = empleado
self.datos_solicitud['legajo'] = legajo
self._reiniciar_intentos('legajo')

self._mostrar_datos_empleado(empleado)

if empleado['dias'] == 0:
    print("\n❌ No tienes días disponibles para solicitar vacaciones.")
    print("📄 Proceso finalizado.")
    return

self._procesar_fecha()
break

def _procesar_fecha(self):
    """Procesa el ingreso de la fecha"""
    self.estado_actual = self.ESTADO_SOLICITANDO_FECHA

    while True:
        print("\n📅 Ingresar la fecha de inicio (formato DD/MM/AAAA):")
        fecha_input = input("📅 Fecha: ").strip()

        if self._verificar_salida(fecha_input):
            return

        resultado = FechaValidator.validate(fecha_input)

        if not resultado.valid:
            self._incrementar_intentos('fecha')
            print(f"\n🚫 {resultado.message}")

            if self.intentos.get('fecha', 0) >= self.MAX_INTENTOS:
                print("\n❌ Demasiados intentos fallidos. El proceso ha sido cancelad
o.")

                sys.exit(0)
                continue

        self.datos_solicitud['fecha_inicio'] = resultado.data['fecha_str']
        self.datos_solicitud['fecha_obj'] = resultado.data['fecha']
        self._reiniciar_intentos('fecha')

        self._procesar_dias()
        break

```

```

def _procesar_dias(self):
    """Procesa el ingreso de días"""
    self.estado_actual = self.ESTADO_SOLICITANDO_DIAS

    while True:
        print("\n📅 ¿Cuántos días de vacaciones deseas solicitar?")
        print(f"💡 Días disponibles: {self.datos_solicitud['empleado']['dias']}")
        dias_input = input("📅 Días: ").strip()

        if self._verificar_salida(dias_input):
            return

        resultado = DiasValidator.validate(
            dias_input,
            self.datos_solicitud['empleado']['dias']
        )

        if not resultado.valid:
            self._incrementar_intentos('dias')
            print(f"\n🚫 {resultado.message}")

            if self.intentos.get('dias', 0) >= self.MAX_INTENTOS:
                print("\n❌ Demasiados intentos fallidos. El proceso ha sido cancelad
o.")
                sys.exit(0)
                continue

            self.datos_solicitud['dias_solicitados'] = resultado.data['dias']
            self._reiniciar_intentos('dias')

            self._evaluar_solicitud()
            break

def _evaluar_solicitud(self):
    """Evalúa la solicitud y decide el siguiente paso"""
    self.estado_actual = self.ESTADO_EVALUANDO

    dias = self.datos_solicitud['dias_solicitados']
    empleado = self.datos_solicitud['empleado']

    print(f"\n🔄 Procesando solicitud de {dias} días...")

    # Evaluar si requiere aprobación del supervisor
    if dias > self.DIAS_APROBACION_AUTOMATICA:
        print(f"\n⚠️ La solicitud de {dias} días requiere aprobación del superviso
r.")
        self._procesar_aprobacion_supervisor()
    else:
        print(f"\n✅ Solicitud de {dias} días aprobada automáticamente.")
        self._aprobar_solicitud()

def _procesar_aprobacion_supervisor(self):
    """Procesa la aprobación del supervisor"""
    self.estado_actual = self.ESTADO_APROBACION_SUPERVISOR

    while True:
        print("\n👤 Supervisor, ¿desea aprobar esta solicitud?")
        print("    S → Aprobar")
        print("    N → Rechazar")
        respuesta = input("👤 (S/N): ").strip().upper()

        if self._verificar_salida(respuesta):
            return

        if respuesta == "S":
            print("\n✅ Solicitud aprobada por el supervisor.")
            self._aprobar_solicitud()

```

```

        break

    if respuesta == "N":
        print("\n❌ Solicitud rechazada por el supervisor.")
        self._rechazar_solicitud("Rechazada por supervisor")
        break

    print("❌ Respuesta inválida. Ingresar 'S' o 'N'.")

def _aprobar_solicitud(self):
    """Aprueba la solicitud y actualiza la base de datos"""
    try:
        empleado = self.datos_solicitud['empleado']
        legajo = empleado['legajo']
        dias = self.datos_solicitud['dias_solicitados']
        fecha = self.datos_solicitud['fecha_inicio']

        saldo_anterior = empleado['dias']
        saldo_nuevo = saldo_anterior - dias

        # Actualizar saldo en Excel
        self.excel_manager.actualizar_dias_empleado(legajo, saldo_nuevo)

        # Registrar solicitud
        self.excel_manager.registrar_solicitud(
            legajo=legajo,
            nombre=empleado['nombre'],
            sector=empleado['sector'],
            fecha_inicio=fecha,
            dias=dias,
            estado="Aprobada"
        )

        self._mostrar_comprobante(
            estado="Aprobada",
            saldo_anterior=saldo_anterior,
            saldo_actual=saldo_nuevo
        )

        self._finalizar()

    except Exception as e:
        print(f"\n❌ Error al procesar la solicitud: {e}")
        print("📄 Por favor, contacta al administrador.")
        logger.error(f"Error en aprobación: {e}")

def _rechazar_solicitud(self, motivo: str = "Rechazada"):
    """Rechaza la solicitud"""
    try:
        empleado = self.datos_solicitud['empleado']

        # Registrar solicitud rechazada
        self.excel_manager.registrar_solicitud(
            legajo=empleado['legajo'],
            nombre=empleado['nombre'],
            sector=empleado['sector'],
            fecha_inicio=self.datos_solicitud['fecha

```